



Flutter Application

Dart Language
Google Product

Flutter - Installation

- Android Studio
- Visual Studio Code
- Power Shell prompt
- Pub Get Flutter
- Flutter Doctor

Flutter - Introduction

- **What is Flutter**

Originally developed by Google and now managed by ECMA,

Flutter offers free and open-source UI development support.

With one programming language and one codebase, users can create beautiful, natively compiled mobile applications with this UI toolkit.

A framework like Flutter allows you to build mobile applications for iOS and Android in a truly platform-independent way.

Developers consider it to be the fastest and most expressive way to build native apps.

Due to its simplicity, high performance resulting from its development, rich user interface, Flutter will have a significant impact on the development of high-quality, feature-packed mobile applications in the near future.

Flutter - Introduction

- ❖ Flutter is Google's UI toolkit:
 - for building beautiful, natively compiled applications
 - for mobile, web, and desktop from a single codebase.
- ❖ Flutter works:
 - with existing code,
 - is used by developers and organizations around the world,
 - and is free
 - and open source.
- ❖ Flutter is an open source framework by Google for building beautiful, natively compiled, multi-platform applications from a single codebase.

Flutter - Introduction

- ❖ **Flutter widgets** are built using a modern framework that takes inspiration from [React](#).
- ❖ The **central idea** is that you build your UI out of widgets.
- ❖ Flutter is the framework for **cross-platform** applications
- ❖ Flutter is one of the **best solutions** to:
 - develop apps for Android and iOS,
 - without having to write in a different codebase for each platform.
 - The smartphone versions of these apps function as true,
 - native apps on Apple and Android devices
 - and are compiled for the respective platform before publication.
 - They do not need a runtime module or a browser.
 - Using the same codebase, it is also possible to create web apps for browsers
 - as well as native programs for Windows, Linux and macOS.

Apps developed on Flutter

- The popularity of Flutter app development is constantly growing.
- Now Flutter is widely used to create apps for the likes of Alibaba, Yandex, Airbnb, Uber, eBay and other leading companies.



Pros of Flutter app development

1. One codebase for all platforms

- Gone are the days of having to write a code for Android and another codebase for iOS devices.
- Flutter's code reusability allows you to write just one codebase and use it on not only for mobile Android and iOS but even for web, desktop and more.
- This cuts development time significantly, removes cost and enables you launch your app that much faster.

2. “It’s all Widgets” principle offers countless possibilities

- Flutter's custom widgets are an absolute delight when it comes to creating great visuals for your app.
- At the same time, you don't have to worry about the UI on different devices.

3. Rich libraries

- Flutter uses the Skia Graphics Library which is a fast and mature open-source graphics library.
- It redraws the UI every time a view changes.
- The result? A quick loading and smooth app experience.

4. Fast testing with hot reload

- The hot reload feature make the app development much quicker.
- With Flutter there is no need to reload the app to see every single change you make in the code.
- You can easily make changes in your app in real time, so you have more opportunity to experiment with the code and fix bugs on the go.

Cons of Flutter app development

1. Large file size because of the widgets

- Flutter apps are quite large and “heavy” to start with.
- They occupy a lot of space and take longer to download or update.

2. Complex updating

- Updating programming requirements in operating systems requires updating Flutter modules.
- Since the modules are integrated as fixed elements in the program,
- the latter must also be recompiled and reinstalled on the devices.

3. Limited set of tools and libraries

- Flutter is a quite new development framework,
- so in some cases you may not be able to get the desired functions in the current library.
- Flutter will take some time to create certain tools, expand functionality and develop the community.

- <https://www.appify.digital/post/flutter-app-development>

Best Text Editors & IDE For Flutter App Development

- ❖ VS Code
- ❖ Android Studio
- ❖ IntelliJ IDEA
- ❖ Sublime Text
- ❖ Emacs

References: <https://www.intelivita.com/blog/best-ide-text-editors-for-flutter/>

Flutter

- ❖ **Flutter** is an open-source UI toolkit for creating exceptional front-end cross-platform applications.
- ❖ **Flutter** is a mobile app development framework.
- ❖ **Flutter** support desktop application development.
- ❖ **Flutter** is developed by Google.
- ❖ Flutter programs are written in **Google's** own [Dart programming language](#).
- ❖ The **features of Flutter** are :
 - Fast development
 - Huge widget catalog
 - High performance application

Flutter

- ❖ **pubspec.yaml** lives at the base of a Flutter project and provides the necessary configuration and setup to build Flutter apps.
- ❖ **Keys** are used in Flutter to uniquely identify widgets, elements, and SemanticsNodes.
- ❖ **Firebase database** allows for access to a cloud-based database provider for Flutter applications.
- ❖ A sequence of asynchronous events is often referred to as a **stream**.
- ❖ The purpose of the **runApp()** function in Flutter is to start the Flutter application.
- ❖ The purpose of the **MaterialApp** widget in Flutter is to provide a set of material design widgets.
- ❖ Unlike other programming languages Dart **doesn't support arrays**.
- ❖ The purpose of the **Flutter Engine** is to compile dart code to machine code.

widget in Flutter Application

- ❖ **SizedBox** component allow us to specify the distance between widgets on the screen.
- ❖ A **widget in Flutter** is a visual element in the user interface.
- ❖ **Widgets** describe what their view should look like given their current configuration and state.
- ❖ There are **two types of widgets** available to developers in Flutter (stateful and stateless widgets).
- ❖ **Stateful widgets** allow you to modify their data in response to user input or retrieving data.
- ❖ The **GridView** widget is used to create a grid in Flutter. It allows the user to display a collection of widgets in a grid layout and provides various customization options.
- ❖ The purpose of the **Navigator widget** in Flutter is to navigate between different screens in the app.

widget in Flutter Application

- The **Container widget** in Flutter is used to provide a flexible container for layout and styling. It can be used to add padding, margins, and borders to other widgets, as well as to apply background and foreground colors.
- The **Scaffold widget** in Flutter is used to provide a layout for the application's pages. It can be used to add various material design elements, such as app bars, drawers, and bottom navigation, to the application's pages.

Type of widget

- ❖ There are **two types of widgets** available to developers in Flutter (stateful and stateless widgets).
- ❖ **Stateful widgets** allow you to modify their data in response to user input or retrieving data.
- ❖ The **StatefulWidget** in Flutter is used to manage the state of the application. It can be used to create widgets that need to be updated based on user input or any other state change.
- The **StatelessWidget** in Flutter is used to build a widget tree that does not depend on any mutable state. It can be used to create widgets that do not need to be updated based on user input or any other state change.

The Minimal Stateless app

```
1 import 'package:flutter/widgets.dart';
2
3 main() => runApp(MyApp());
4
5 class MyApp extends StatelessWidget {
6   @override
7   Widget build(context) => Center(
8     child: Text('Hello Flutter!', textDirection: TextDirection.ltr)
9   );
10 }
```

```
1 import 'package:flutter/material.dart';
2
3 main() => runApp(MyApp());
4
5 class MyApp extends StatelessWidget {
6   @override
7   Widget build(context) {
8     return MaterialApp(
9       home: Scaffold(
10        body: Center(
11          child: Text('Hello World'),
12        ),
13      ),
14    );
15  }
16 }
```

The Minimal *stateful* app

```
3  main() => runApp(MinimalStatefulApp());
4
5  class MinimalStatefulApp extends StatefulWidget {
6    @override
7    _MinimalState createState() => _MinimalState();
8  }
9
10 class _MinimalState extends State<MinimalStatefulApp> {
11
12   int _counter = 0;
13
14   @override
15   Widget build(BuildContext context) {
16     return GestureDetector(
17       onTap: () => setState(() => _counter++),
18       child: Center(
19         child: Text(
20           'Counter: $_counter',
21           textDirection: TextDirection.ltr,
22         ),
23       ),
24     );
25   }
26 }
```


Basic widgets

Flutter comes with a suite of powerful basic widgets, of which the following are commonly used:

Text

The `Text` widget lets you create a run of styled text within your application.

Row, Column

These flex widgets let you create flexible layouts in both the horizontal (`Row`) and vertical (`Column`) directions. The design of these objects is based on the web's flexbox layout model.

Stack

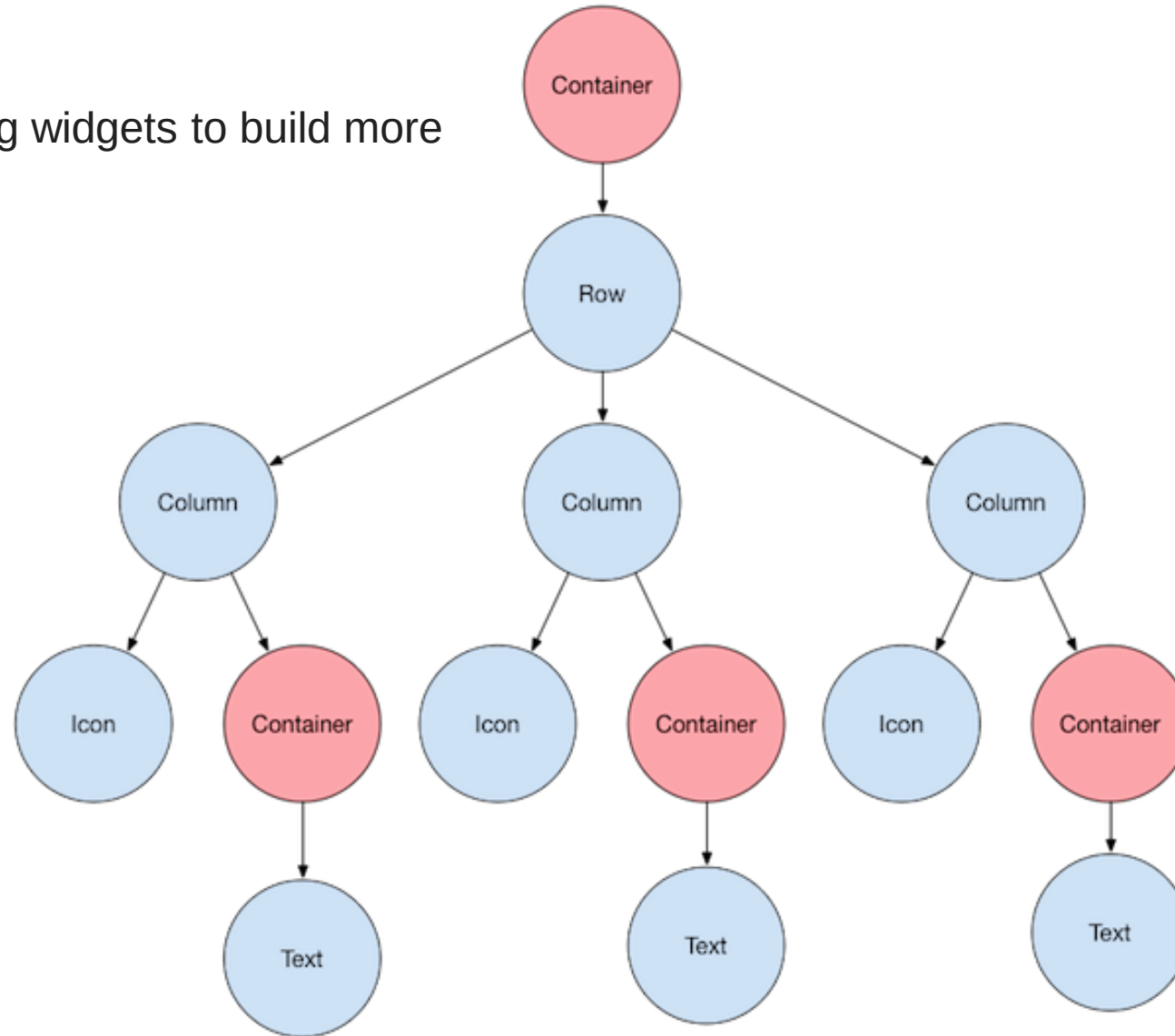
Instead of being linearly oriented (either horizontally or vertically), a `Stack` widget lets you place widgets on top of each other in paint order. You can then use the `Positioned` widget on children of a `Stack` to position them relative to the top, right, bottom, or left edge of the stack. Stacks are based on the web's absolute positioning layout model.

Container

The `Container` widget lets you create a rectangular visual element. A container can be decorated with a `BoxDecoration`, such as a background, a border, or a shadow. A `Container` can also have margins, padding, and constraints applied to its size. In addition, a `Container` can be transformed in three-dimensional space using a matrix.

Flutter - Introduction to Widgets ... cont

You create a layout by composing widgets to build more complex widgets. For example



Widget Build Visualization

In Flutter, widgets can be grouped into multiple categories based on their features, as listed below –

- Platform specific widgets
- Layout widgets
- State maintenance widgets
- Platform independent / basic widgets

Let us discuss each of them in detail now.

Platform specific widgets

Flutter has widgets specific to a particular platform - Android or iOS.

Android specific widgets are designed in accordance with Material design guideline by Android OS. Android specific widgets are called as Material widgets.

iOS specific widgets are designed in accordance with Human Interface Guidelines by Apple and they are called as Cupertino widgets.

Some of the most used material widgets are as follows –

- Scaffold
- AppBar
- BottomNavigationBar
- TabBar
- TabBarView
- ListTile
- RaisedButton
- FloatingActionButton
- FlatButton
- IconButton
- DropdownButton
- PopupMenuButton
- ButtonBar
- TextField
- Checkbox
- Radio
- Switch
- Slider
- Date & Time Pickers
- SimpleDialog
- AlertDialog

Some of the most used Cupertino widgets are as follows –

- CupertinoButton
- CupertinoPicker
- CupertinoDatePicker
- CupertinoTimerPicker
- CupertinoNavigationBar
- CupertinoTabBar
- CupertinoTabScaffold
- CupertinoTabView
- CupertinoTextField
- CupertinoDialog
- CupertinoDialogAction
- CupertinoFullscreenDialogTransition
- CupertinoPageScaffold
- CupertinoPageTransition
- CupertinoActionSheet
- CupertinoActivityIndicator
- CupertinoAlertDialog
- CupertinoPopupSurface
- CupertinoSlider

Layout widgets

In Flutter, a widget can be created by composing one or more widgets. To compose multiple widgets into a single widget, Flutter provides large number of widgets with layout feature. For example, the child widget can be centered using Center widget.

Some of the popular layout widgets are as follows –

- **Container** – A rectangular box decorated using BoxDecoration widgets with background, border and shadow.
- **Center** – Center its child widget.
- **Row** – Arrange its children in the horizontal direction.
- **Column** – Arrange its children in the vertical direction.
- **Stack** – Arrange one above the another.

State maintenance widgets

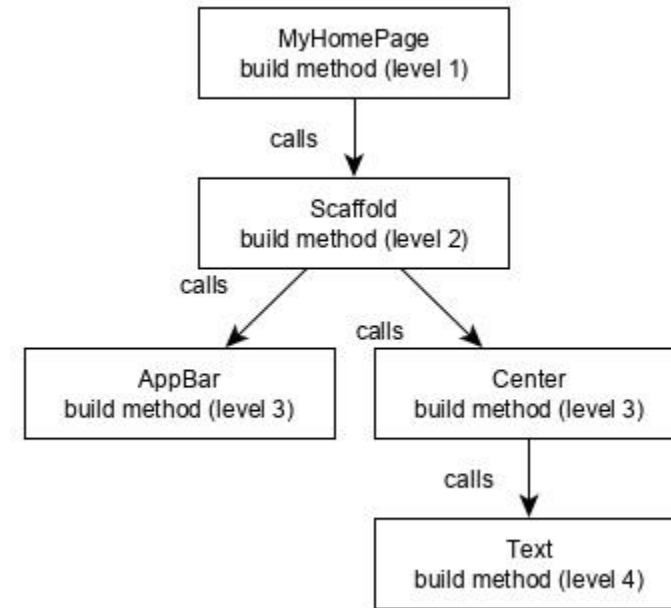
- In Flutter, all widgets are either derived from:
StatelessWidget or *StatefulWidget*.

Platform independent / basic widgets

- *Flutter* provides large number of basic widgets to create simple
- as well as complex user interface in a platform independent manner.

Flutter - Introduction to Widgets ... cont

```
class MyHomePage extends StatelessWidget {  
  MyHomePage({Key key, this.title}) : super(key: key);  
  
  final String title;  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(title: Text(this.title), ),  
      body: Center(child: Text( 'Hello World',)),  
    );  
  }  
}
```



- *In the code, we have used title as one of the constructor argument and also used Key as another argument.*
- *The title is used to display the title and Key is used to identify the widget in the build environment.*
- *The title is used to display the title and Key is used to identify the widget in the build environment.*
- *Here, the build method calls the build method of Scaffold, which in turn calls the build method of AppBar and Center to build its user interface.*
- *Finally, Center build method calls Text build method.*

Flutter Text (display a string)

- A Text is a widget in Flutter that allows us to **display a string of text with a single line in our application.**
- Depending on the layout constraints, we can break the string across multiple lines or might all be displayed on the same line.

The most important properties of the *Text* widget are as follows:

- **maxLines, int** – Maximum number of lines to show
- **overflow, TextOverflow** – Specify how visual overflow is handled using *TextOverflow* class
- **style, TextStyle** – Specify the style of the string using *TextStyle* class
- **textAlign, TextAlign** – Alignment of the text like right, left, justify, etc., using *TextAlign* class
- **textDirection, TextDirection** – Direction of text to flow, either left-to-right or right-to-left

```
body: Center(  
  child:Text(  
    "Hello World! This is a Text Widget.",  
    style: TextStyle(  
      fontSize: 35,  
      color: Colors.purple,  
      fontWeight: FontWeight.w700,  
      fontStyle: FontStyle.italic,  
      letterSpacing: 8,  
      wordSpacing: 20,  
      backgroundColor: Colors.yellow,  
      shadows: [  
        Shadow(color: Colors.blueAccent, offset: Offset(2,1), blurRadius:10)  
      ]  
    ),  
  ),  
)
```


The following are the essential properties of the Text widget used in our application:

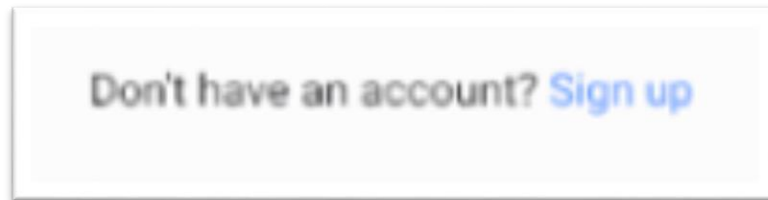
- **TextAlign:** It is used to specify how our text is aligned horizontally. It also controls the text location.
- **TextDirection:** It is used to determine how textAlign values control the layout of our text. Usually, we write text from left to right, but we can change it using this parameter.
- **Overflow:** It is used to determine when the text will not fit in the available space. It means we have specified more text than the available space.
- **TextScaleFactor:** It is used to determine the scaling to the text displayed by the Text widget. Suppose we have specified the text scale factor as 1.5, then our text will be 50 percent larger than the specified font size.
- **SoftWrap:** It is used to determine whether or not to show all text widget content when there is not enough space available. If it is true, it will show all content. Otherwise, it will not show all content.
- **MaxLines:** It is used to determine the maximum number of lines displayed in the text widget.
- **TextWidthBasis:** It is used to control how the text width is defined.
- **TextHeightBehavior:** It is used to control how the paragraph appears between the first line and descent of the last line.
- **Style:** It is the most common property of this widget that allows developers to styling their text. It can do styling by specifying the foreground and background color, font size, font weight, letter and word spacing, locale, shadows, etc. See the table to understand it more easily:

Attributes	Descriptions
foreground	It determines the paint as a foreground for the text.
background	It determines the paint as a background for the text.
fontWeight	It determines the thickness of the text.
fontSize	It determines the size of the text.
fontFamily	It is used to specify the typeface for the font. For this, we need to download a typeface file in our project, and then keep this file into the assets/font folder. Finally, config the pubspec.yaml file to use it in the project.
fontStyle	It is used to style the font either in bold or italic form.
Color	It is used to determine the color of the text.
letterSpacing	It is used to determine the distance between the characters of the text.
wordSpacing	It is used to specify the distance between two words of the text.
shadows	It is used to paint underneath the text.
decoration	We use this to decorate text using the three parameters: decoration, decorationColor, decorationStyle. The decoration determines the location, decorationColor specify the color, decorationStyle determine the shape.

```
child:Text(
  "Hello World! This is a Text Widget.",
  style: TextStyle(
    fontSize: 35,
    color: Colors.purple,
    fontWeight: FontWeight.w700,
    fontStyle: FontStyle.italic,
    letterSpacing: 8,
    wordSpacing: 20,
    backgroundColor: Colors.yellow,
    shadows: [
      Shadow(color: Colors.blueAccent, offset: Offset(2,1), blurRadius:10
    ]
  ),
)
```



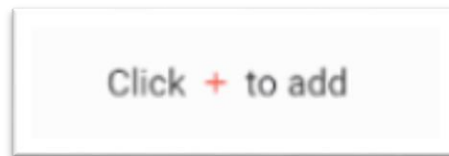
```
body: Container(
  child: Center(
    child: RichText(
      text: TextSpan(
        text: 'Don\'t have an account?',
        style: TextStyle(color: Colors.black, fontSize: 20),
        children: <TextSpan>[
          TextSpan(text: ' Sign up',
            style: TextStyle(color: Colors.blueAccent, fontSize: 20),
            recognizer: TapGestureRecognizer()
              ..onTap = () {}
          )
        ]
      ),
    ),
  ),
)
```



```

body: Container(
  child: Center(
    child: RichText(
      text: TextSpan(
        style: Theme.of(context).textTheme.body1,
        children: [
          TextSpan(text: 'Click ', style: TextStyle(fontSize: 25)),
          WidgetSpan(
            child: Padding(
              padding: const EdgeInsets.symmetric(horizontal: 2.0),
              child: Icon(Icons.add, color: Colors.red),
            ),
          ),
          TextSpan(text: ' to add', style: TextStyle(fontSize: 25)),
        ],
      ),
    ),
  ),
)

```



Flutter TextField Input

Flutter TextFormField forms

Two types of input widgets:

- TextField in Flutter is the most commonly used **text input widget** that allows users to collect inputs from the keyboard into an app.
- TextFormField, a slightly more advanced version of TextField.

TextFormField

- provides more functionalities than TextField, such as build form validation and the ability to set initial text value directly.
- If your text field requires only one or two inputs from the user, I suggest using the TextField widget.
- Otherwise if you want to create a larger form with multiple input fields and validation, it's better to go with the TextFormField widget.

TextField widget

- [TextField](#) is the most commonly used text input widget.
- By default, a [TextField](#) is decorated with an underline.
- You can add a label, icon, inline hint text, and error text by supplying an [InputDecoration](#) as the [decoration](#) property of the [TextField](#).
- To remove the decoration entirely (including the underline and the space reserved for the label), set the [decoration](#) to null.
- **decoration:** It is used to show the decoration around TextField.
- **border:** It is used to create a default rounded rectangle border around TextField.
- **labelText:** It is used to show the label text on the selection of TextField.
- **hintText:** It is used to show the hint text inside TextField.
- **icon:** It is used to add icons directly to the TextField.

TextField properties

```
body: Container(  
  alignment: Alignment.center,  
  padding: EdgeInsets.all(10.0),  
  child: TextField(  
    controller: _usernameController,  
    decoration: InputDecoration(  
      border: OutlineInputBorder(  
        borderRadius: BorderRadius.all(Radius.circular(15)),  
      ),  
      hoverColor: Colors.blueAccent,  
      hintText: 'Enter Your Username',  
      prefixIcon: Icon(Icons.account_circle),  
      labelText: 'Username',  
    ),  
    textAlign: TextAlign.left,  
    keyboardType: TextInputType.text,  
    maxLength: 15,  
    style: TextStyle(  
      color: Colors.black,  
      fontSize: 12.0,  
      fontStyle: FontStyle.italic),  
    onSubmitted: (value) {  
      getData();  
    }  
  ));
```

```
import 'package:flutter/material.dart';  
  
// function to instantiate the app  
void main() => runApp(MyApp());  
  
class MyApp extends StatelessWidget {  
  final fieldText = TextEditingController();  
  
  MyApp({Key? key}) : super(key: key);  
  
  void clearText() {  
    fieldText.clear();  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        appBar: AppBar(  
          title: const Text('GeeksforGeeks'),  
          backgroundColor: Colors.green,  
        ),  
        body: Center(  
          // Use SizedBox for whitespace  
  
          child: SizedBox(  
            width: 250,  
            child: TextField(  
              decoration: InputDecoration(  
                hintText: 'Enter Something',  
                focusColor: Colors.green,  
                suffixIcon: IconButton(  
                  icon: const Icon(Icons.clear),  
                  onPressed: clearText,  
                ),  
              ),  
              controller: fieldText,  
            ), // TextField  
          ), // SizedBox  
        ),  
      ),  
    );  
  }  
}
```

TextField

The types are:

- text (Normal complete keyboard)
- number (A numerical keyboard)
- emailAddress (Normal keyboard with an “@”)
- datetime (Numerical keyboard with a “/” and “:”)
- numberWithOptions (Numerical keyboard with options to enable signed and decimal mode)
- multiline (Optimises for multi-line information)

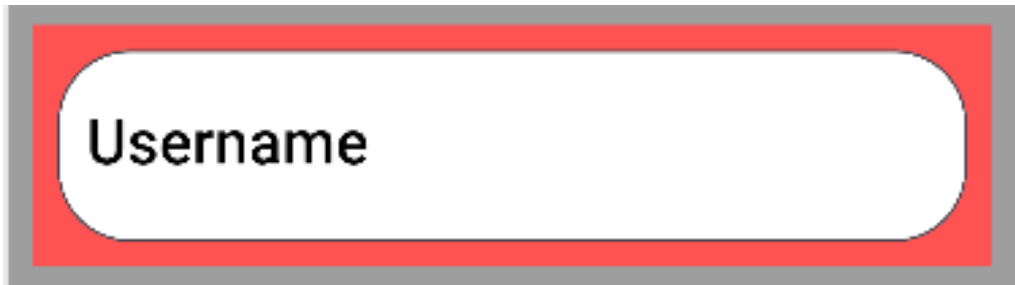
References: <https://blog.logrocket.com/the-ultimate-guide-to-text-fields-in-flutter/>

TextField

```
class MyApp extends StatelessWidget {  
  final userController = TextEditingController();  
  final passController = TextEditingController();  
  final showController = TextEditingController();  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  

```

```
      child: TextField(  
        controller: userController,  
        keyboardType: TextInputType.name,  
        decoration: const InputDecoration(  
          filled: true,  
          //fillColor: Color(0xffF02E65),  
          fillColor: Color.fromARGB(255, 255, 255, 255),  
          // fillColor: Colors.deepPurpleAccent,  
          hintText: 'Username',  
          hintStyle: TextStyle(  
            color: Colors.black,  
            fontSize: 25,  
            // backgroundColor: Colors.white,  
          ), // TextStyle  
          border: OutlineInputBorder(  
            borderRadius: BorderRadius.all(Radius.circular(30.0)),  
          ), // OutlineInputBorder  
        ), // InputDecoration  
      ), // TextField  
    ),  
    //=====  
  ), // Container
```

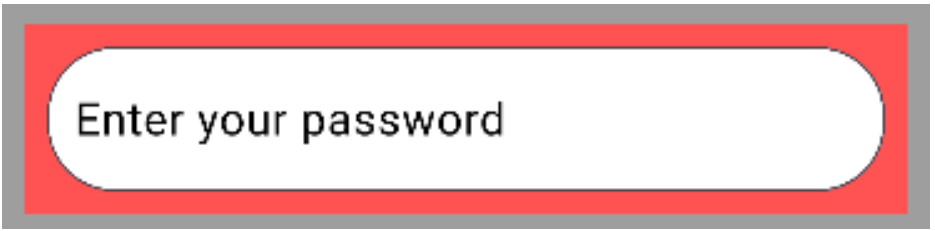


TextFormField

```
class MyApp extends StatelessWidget {  
  final userController = TextEditingController();  
  final passController = TextEditingController();  
  final showController = TextEditingController();  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Scaffold(  
        //=====  
        appBar: AppBar(  
          backgroundColor: Colors.green,  
          title: Text('Login Page'),  
        ), // AppBar  
        backgroundColor: Colors.grey,  
        body: Column(  
          children: [  

```

```
Container(  
  margin: EdgeInsets.all(10.0),  
  padding: EdgeInsets.all(10.0),  
  color: Colors.redAccent,  
  //=====  
  child: TextFormField(  
    controller: passController,  
    decoration: const InputDecoration(  
      labelText: 'Enter your password',  
      labelStyle: TextStyle(  
        color: Colors.black,  
        fontSize: 20,  
      ), // TextStyle  
      fillColor: Colors.white,  
      border: OutlineInputBorder(  
        borderRadius: BorderRadius.all(Radius.circular(30.0)),  
      ), // OutlineInputBorder  
    ), // InputDecoration  
  ), // TextFormField  
), // Container
```



Enter your password

Flutter Button

- Buttons are the graphical control element that **provides a user to trigger an event** such as taking actions, making choices, searching things, and many more. They can be placed anywhere in our UI like dialogs, forms, cards, toolbars, etc.
- Buttons are the Flutter widgets, which is a part of the material design library. Flutter provides several types of buttons that have different shapes, styles, and features.

Features of Buttons

- The standard features of a button in Flutter are given below:
- We can easily apply themes on buttons, shapes, color, animation, and behavior.
- We can also theme icons and text inside the button.
- Buttons can be composed of different child widgets for different characteristics.

Flat Button

Raised Button

Floating Button

Drop Down Button

Icon Button

Inkwell Button

PopupMenu Button

Outline Button

Flutter Button Types

The different types of buttons available in Flutter

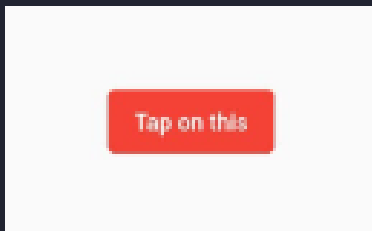
- `ElevatedButton` (buttons such as color, elevation(shadow))
- `OutlinedButton` (buttons such as color & border)
- `TextButton` (buttons such as color)
- `IconButton` (buttons such as color & size)
- `FloatingActionButton` (buttons such as color and position of the FAB)

References: <https://blog.stackademic.com/exploring-flutters-button-widgets-6d27fc077456>

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text("AppMaking.co"),
      centerTitle: true,
      backgroundColor: Colors.blue[900],
    ),
    body: Center(
      child: ElevatedButton(
        child: Text("Tap on this"),
        style: ElevatedButton.styleFrom(
          primary: Colors.red,
          elevation: 0,
        ),
        onPressed: () {},
      ),
    ),
  );
}

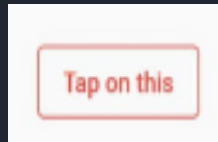
```



```

child: OutlinedButton(
  child: Text("Tap on this"),
  style: OutlinedButton.styleFrom(
    primary: Colors.red,
    side: BorderSide(
      color: Colors.red,
    ),
  ),
  onPressed: () {},
),

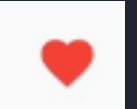
```



```

child: IconButton(
  icon: Icon(Icons.favorite),
  iconSize: 40,
  color: Colors.red,
  onPressed: () {},
),

```



```

child: TextButton(
  child: Text("Tap on this"),
  style: TextButton.styleFrom(
    primary: Colors.blue,
  ),
  onPressed: () {},
),

```



```

floatingActionButtonLocation: FloatingActionButtonLocation.centerFloat,
floatingActionButton: FloatingActionButton(
  backgroundColor: Colors.green,
  child: Icon(Icons.add),
  onPressed: () {},
),

```



Old Buttons

Old Widget	Old Theme	New Widget	New Theme
<code>FlatButton</code>	<code>ButtonTheme</code>	<code>TextButton</code>	<code>TextButtonTheme</code>
<code>RaisedButton</code>	<code>ButtonTheme</code>	<code>ElevatedButton</code>	<code>ElevatedButtonTheme</code>
<code>OutlineButton</code>	<code>ButtonTheme</code>	<code>OutlinedButton</code>	<code>OutlinedButtonTheme</code>

References: <https://docs.flutter.dev/release/breaking-changes/buttons>

Flutter Container

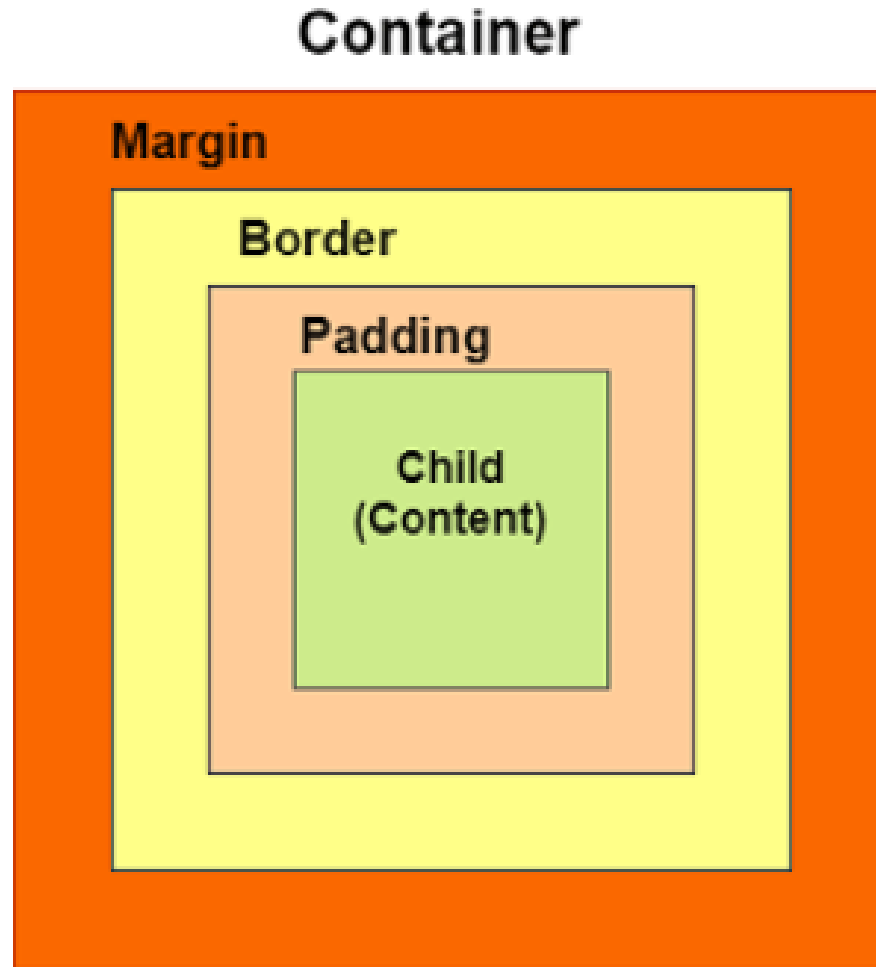


Image location

Several constructors are provided for the various ways that an image can be specified:

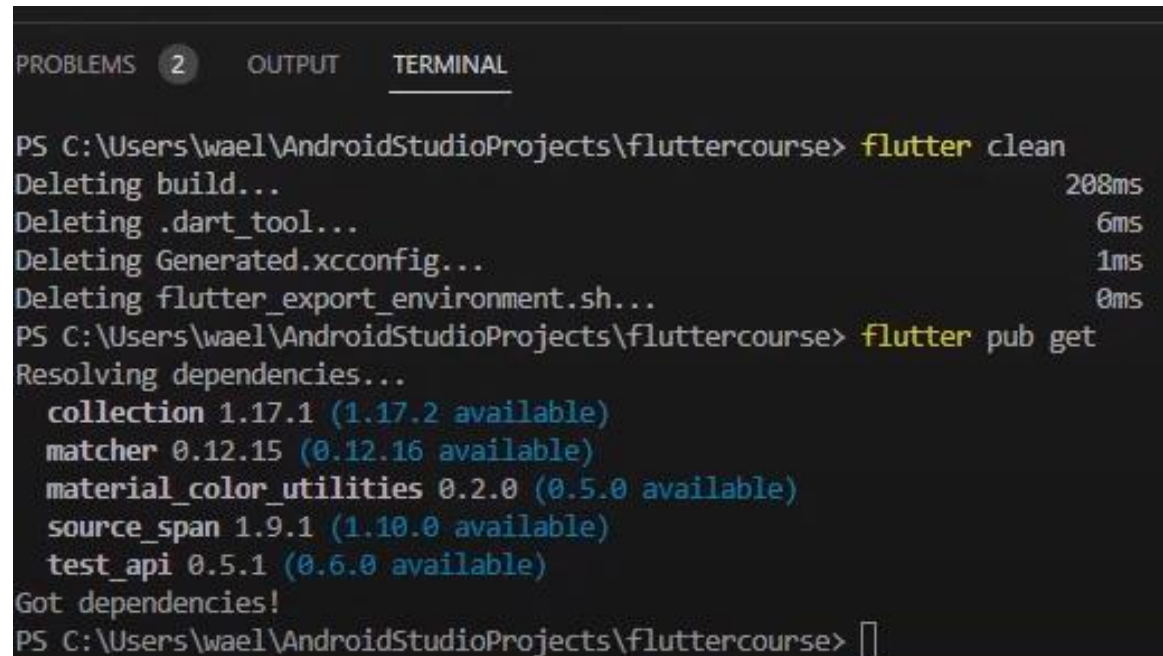
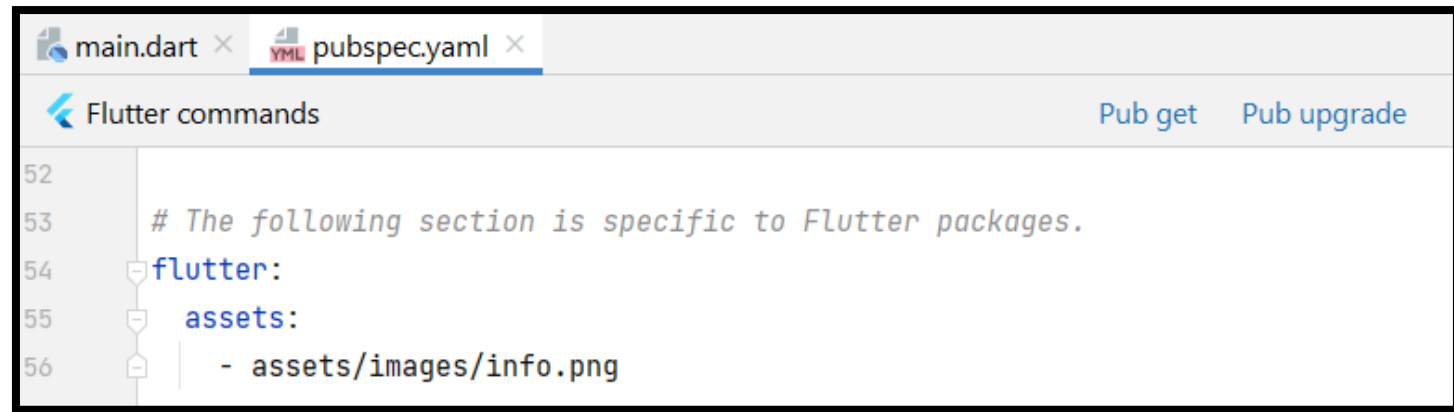
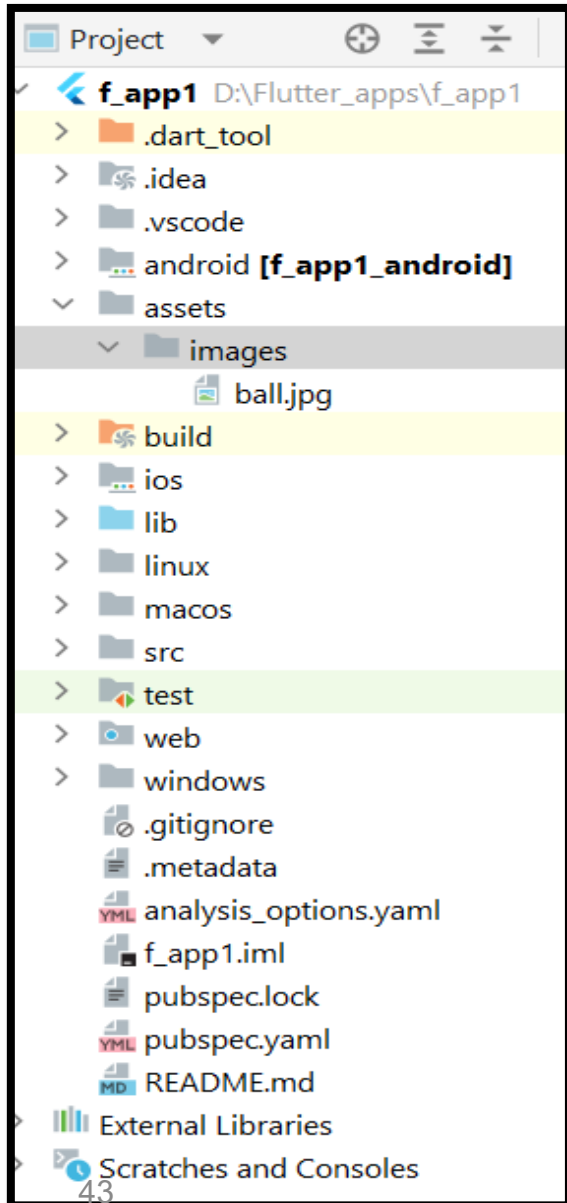
- [Image.new](#), for obtaining an image from an [ImageProvider](#).
- [Image.asset](#), for obtaining an image from an [AssetBundle](#) using a key.
- [Image.network](#), for obtaining an image from a URL.
- [Image.file](#), for obtaining an image from a [File](#).
- [Image.memory](#), for obtaining an image from a [Uint8List](#).

Image location

The following image formats are supported: JPEG, PNG, GIF, Animated GIF, WebP, Animated WebP, BMP, and WBMP.

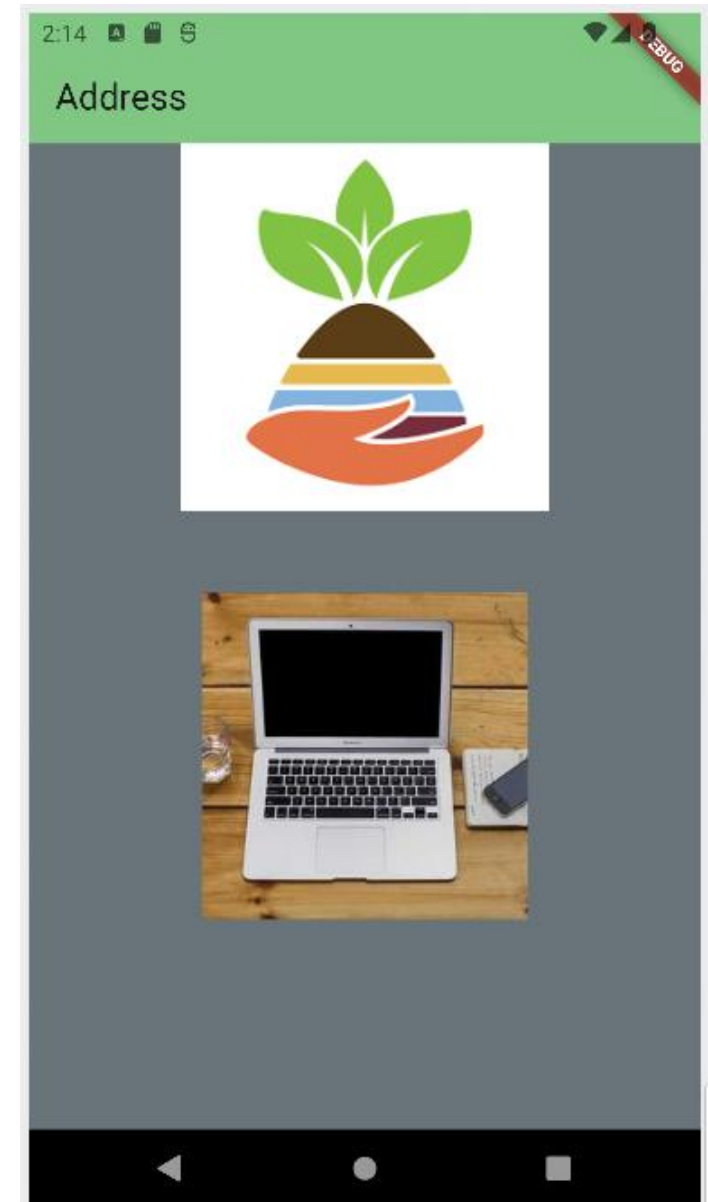
To automatically perform pixel-density-aware asset resolution, specify the image using an [AssetImage](#) and make sure that a [MaterialApp](#), [WidgetsApp](#), or [MediaQuery](#) widget exists above the [Image](#) widget in the widget tree.

Images



Lab3:

```
1 import 'package:flutter/material.dart';
2 void main() {
3   runApp(MaterialApp(
4     home: Scaffold(
5       backgroundColor: Color.fromARGB(255, 104, 116, 122),
6       appBar: AppBar(
7         title: Text('Images'),
8         backgroundColor: Colors.green[300],
9       ), // AppBar
10      body: Center(
11        child: Column(
12          children: <Widget>[
13            Image.asset('assets/images/info.png',
14              width: 400,
15              height: 300,
16              // fit: BoxFit.contain,
17            ), // Image.asset
18            Image.network(
19              'https://picsum.photos/250?image=9',
20              loadingBuilder: (context, child, progress){
21                return progress == null
22                  ?child
23                  : const LinearProgressIndicator(
24                    backgroundColor: Colors.deepPurple,
25                    valueColor: AlwaysStoppedAnimation<Color>(Colors.red,),
26                    value: 0.8,
27                  ); // LinearProgressIndicator
28              },
29              height: 300,
30              width: 200
31            ), // Image.network
32          ], //<Widget>[] // <Widget>[]
33        ), //Column // Column
34      ), //Center // Center
35    ), // Scaffold
36  ), // MaterialApp
37 );
38 }
```



Provide a loadingBuilder
as the images is being download from the network

```
Image.network(  
  'https://example.com/dash.jpg',  
  loadingBuilder: (context, child, progress) {  
    return progress == null  
      ? child  
      : LinearProgressIndicator(...);  
  },  
)
```

The file system has images

```
Image.file(  
  '/path/to/dash.jpg',  
)
```

```
Image.asset(  
  'assets/images/dash.jpg',  
  width: 200,  
  height: 400,  
  fit: BoxFit.contain,  
)
```

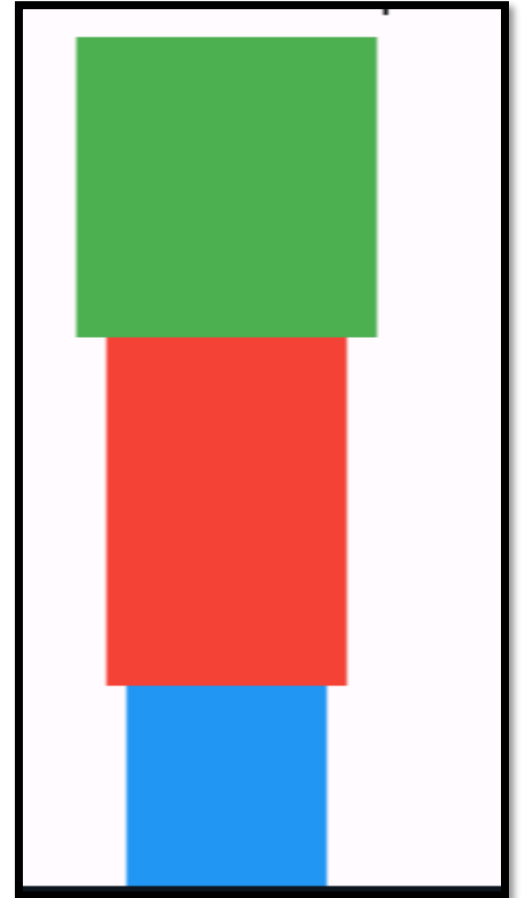
Single-child layout widgets

```
Center(  
  child: Container(  
    height: 120.0,  
    width: 120.0,  
    color: Colors.blue[50],  
    child: const Align(  
      alignment: Alignment.topRight,  
      child: FlutterLogo(  
        size: 60,  
      ),  
    ),  
  ),  
)
```

Alignment.topRight



```
Widget build(BuildContext context) {  
  return Center(  
    child: Column(  
      children: <Widget>[  
        Container(  
          color: Colors.green,  
          height: 150,  
          width: 150,  
        ),  
        Expanded(  
          child: Container(  
            color: Colors.red,  
            width: 120,  
          ),  
        ),  
        Container(  
          color: Colors.blue,  
          height: 100,  
          width: 100,  
        ),  
      ],  
    ),  
  );  
}
```



Single-child layout widgets



```
const SizedBox(  
  width: 200.0,  
  height: 300.0,  
  child: Card(child: Text('Hello World!')),  
)
```

References: <https://docs.flutter.dev/ui/widgets/layout>

Multi-child layout widgets

Column

```
const Column(  
  children: <Widget>[  
    Text('Deliver features faster'),  
    Text('Craft beautiful UIs'),  
    Expanded(  
      child: FittedBox(  
        child: FlutterLogo(),  
      ),  
    ),  
  ],  
)
```

- In the sample above, the text and the logo are centered on each line.
- In the following example, the `crossAxisAlignment` is set to `CrossAxisAlignment.start`, so that the children are left-aligned.
- The `mainAxisSize` is set to `MainAxisSize.min`, so that the column shrinks to fit the children.

```
Column(  
  crossAxisAlignment: CrossAxisAlignment.start,  
  mainAxisSize: MainAxisSize.min,  
  children: <Widget>[  
    const Text('We move under cover and we move as one'),  
    const Text('Through the night, we have one shot to live another day'),  
    const Text('We cannot let a stray gunshot give us away'),  
    const Text('We will fight up close, seize the moment and stay in it'),  
    const Text('It's either that or meet the business end of a bayonet'),  
    const Text('The code word is 'Rochambeau,' dig me?'),  
    Text('Rochambeau!', style: DefaultTextStyle.of(context).style.apply(fontSizeFactor: 2.0)),  
  ],  
)
```

References: <https://docs.flutter.dev/ui/widgets/layout>

Multi-child layout widgets

Row

- A widget that displays its children in a horizontal array.
- To cause a child to expand to fill the available horizontal space, wrap the child in an [Expanded](#) widget.
- The [Row](#) widget does not scroll
- If you have a line of widgets and want them to be able to scroll if there is insufficient room, consider using a [ListView](#).
- For a vertical variant, see [Column](#).

```
const Row(  
  children: <Widget>[  
    Expanded(  
      child: Text('Deliver features faster', textAlign: TextAlign.center),  
    ),  
    Expanded(  
      child: Text('Craft beautiful UIs', textAlign: TextAlign.center),  
    ),  
    Expanded(  
      child: FittedBox(  
        child: FlutterLogo(),  
      ),  
    ),  
  ],  
)
```

Multi-child layout widgets

```
body: Column(  
  children: [  
    Container(  
      child: const TextField( ),  
    ), // Container  
    Container(  
      child: const TextField( ),  
    ), // Container  
    //=====  
    Container(  
      child: Row(  
        children: [  
          Expanded(  
            child: FloatingActionButton(  
              onPressed: () {},  
            ), // FloatingActionButton  
          ), // Expanded  
          ElevatedButton(  
            onPressed: () {},  
            child: Text("Clear"),  
          ), // ElevatedButton  
          Center(  
            child: CloseButton(  
              ),  
            ), // Center
```

```
        Padding(  
          padding: EdgeInsets.fromLTRB(20, 0, 20, 0),  
          child: Container(  
            padding: const EdgeInsets.all(0.0),  
            color: Colors.green,  
            width: 80.0,  
            height: 80.0,  
          ), //Container // Container  
        ), //Padding // Padding  
      ],  
    ), // Row  
  ), // Container  
],  
), // Column  
, // Scaffold  
); // MaterialApp  
}
```

Navigate to a new screen and back

- ❖ In Flutter, *screens* and *pages* are called *routes*.
- ❖ In Android, a route is equivalent to an Activity.
- ❖ In iOS, a route is equivalent to a ViewController.
- ❖ In Flutter, a route is just a widget.

How to navigate between two routes, using these steps:

- Create two routes.
- Navigate to the second route using `Navigator.push()`.
- Return to the first route using `Navigator.pop()`.

1. Create two routes

```
class FirstRoute extends StatelessWidget {
  const FirstRoute({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('First Route'),
      ),
      body: Center(
        child: ElevatedButton(
          child: const Text('Open route'),
          onPressed: () {
            // Navigate to second route when tapped.
          },
        ),
      ),
    );
  }
}
```

```
class SecondRoute extends StatelessWidget {
  const SecondRoute({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Second Route'),
      ),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            // Navigate back to first route when tapped.
          },
          child: const Text('Go back!'),
        ),
      ),
    );
  }
}
```

2. Navigate to the second route using `Navigator.push()`

```
// Within the `FirstRoute` widget
onPressed: () {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => const SecondRoute()),
  );
}
```

3. Return to the first route using `Navigator.pop()`

```
// Within the SecondRoute widget
onPressed: () {
  Navigator.pop(context);
}
```

References: <https://docs.flutter.dev/cookbook/navigation/navigation-basics>

Pass data from PageOne to PageThird by using constructor

```
), // Center
Center(
  child: ElevatedButton(child: Text('Move to Third Page'),
    onPressed: (){
      Navigator.push(context, MaterialPageRoute(builder: (context,)=>
        ThirdPage(message: message))); // MaterialPageRoute
    },
  ),
```

Receiving value in ThirdPage

```
class ThirdPage extends StatelessWidget {
  String message;
  //Student student= Student(name: 'name', ID: 22, gpa: 2);
  ThirdPage({required this.message});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Passing data through constructor')),
      //...
      body: Center(
        child: Text('$message'),
      ), // Center
    );
  }
}
```

Navigate with named routes

- **Note** Named routes are no longer recommended for most applications. For more information, see [Limitations](#) in the [navigation overview](#) page.

References: <https://docs.flutter.dev/cookbook/navigation/named-routes>

Pass data from PageOne to PageTwo

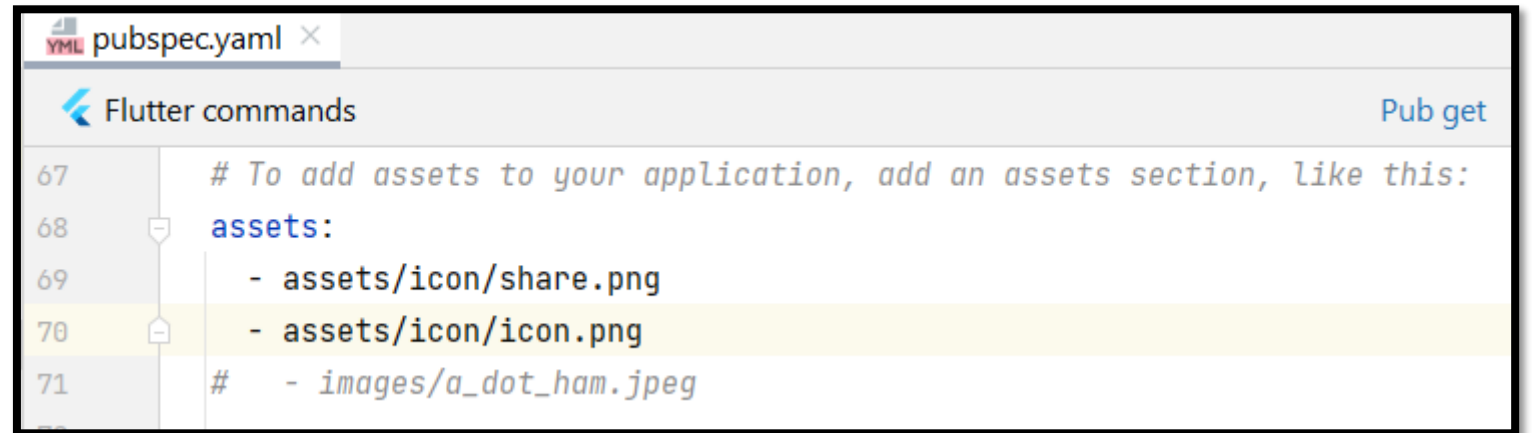
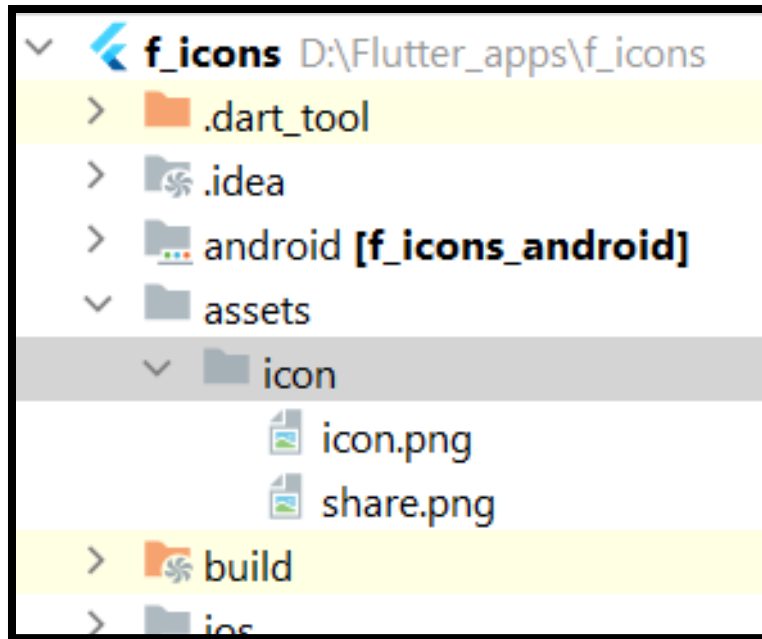
```
class PageOne extends StatelessWidget {  
  
  String message = 'This is message from First Page';  
  //...  
  @override  
  Widget build(BuildContext context) {  
    // String m= 'This is a message from Page one';  
    return Scaffold(  
      appBar: AppBar(  
        title: Text("Page One"),  
      ), // AppBar  
      body: Center(  
        child: ElevatedButton(child: Text('Move to Second Page'),  
          onPressed: (){  
            Navigator.pushNamed(context, PageTwo.page, arguments: message);  
          },  
        ), // ElevatedButton  
      ), // Center  
    );  
    //...  
  }  
}
```

Receiving value in SecondPage ... cont

```
class PageTwo extends StatelessWidget {  
  static String page = "second";  
  //...  
  
  @override  
  Widget build(BuildContext context) {  
    // var data = ModalRoute.of(context)!.settings.arguments as Student;  
    var msg = ModalRoute.of(context)!.settings.arguments as String;  
    return Scaffold(  
      appBar: AppBar(  
        title: Text("Second Page"),  
      ), // AppBar  
      body: Center(  
        child: Text('$msg', style: TextStyle(fontSize: 25),),  
      ), // Center  
    );  
  }  
}
```

Change Flutter App Icons

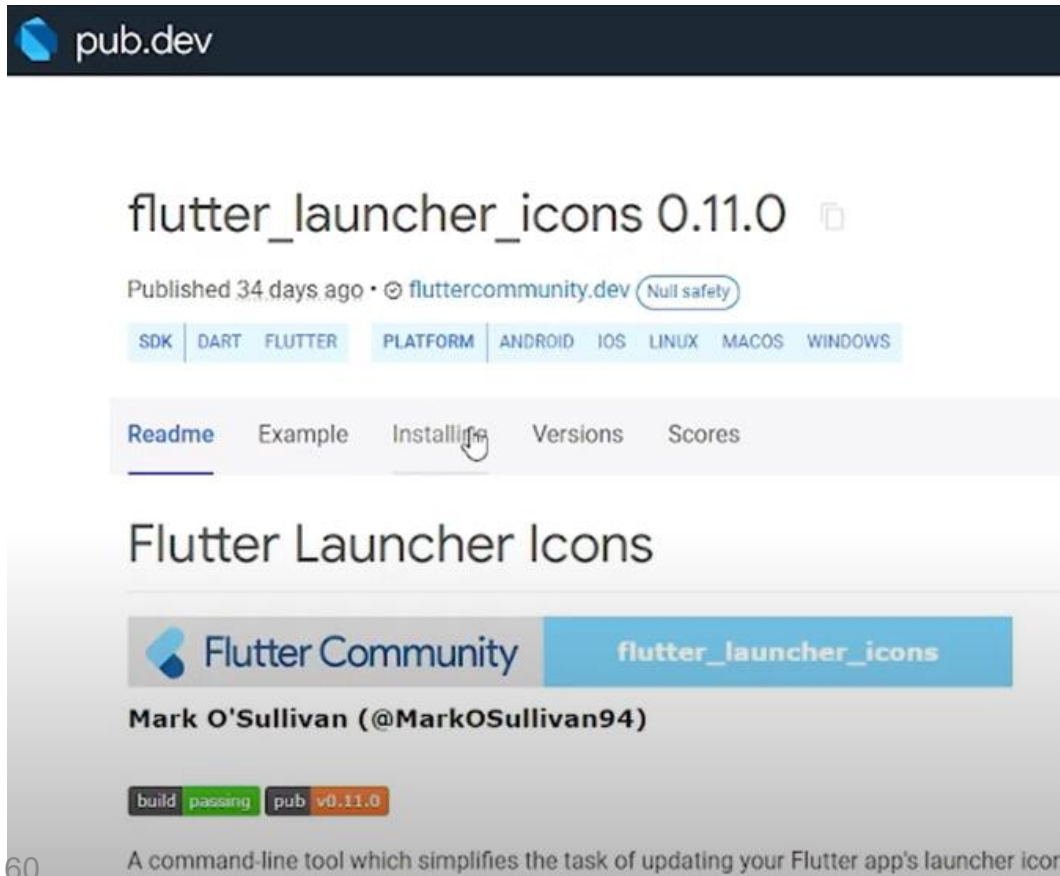
Step #1: Add icon to directory



Change Flutter App Icons Flutter ... cont

Step #2: Open Link then flow steps

https://pub.dev/packages/flutter_launcher_icons



The screenshot shows the pub.dev package page for `flutter_launcher_icons` version 0.11.0. The page header includes the pub.dev logo and the package name. Below the name, it states "Published 34 days ago" and "fluttercommunity.dev" with a "Null safety" badge. A navigation bar contains links for SDK, DART, FLUTTER, PLATFORM, ANDROID, IOS, LINUX, MACOS, and WINDOWS. Another navigation bar includes links for Readme, Example, Install, Versions, and Scores. The main heading is "Flutter Launcher Icons". Below this, there is a Flutter Community logo and the package name in a blue box. The author is listed as "Mark O'Sullivan (@MarkOSullivan94)". There are badges for "build passing" and "pub v0.11.0". At the bottom, a description reads: "A command-line tool which simplifies the task of updating your Flutter app's launcher icon."



The screenshot shows the "Guide" section of the `flutter_launcher_icons` package. It starts with the heading "1. Setup the config file". The text explains that users should add their Flutter Launcher Icons configuration to their `pubspec.yaml` or create a new config file called `flutter_launcher_icons.yaml`. An example is shown below, with more complex examples available in the package's example projects. The example code is as follows:

```
dev_dependencies:
  flutter_launcher_icons: "^0.13.1"

flutter_launcher_icons:
  android: "launcher_icon"
  ios: true
  image_path: "assets/icon/icon.png"
  min_sdk_android: 21 # android min sdk min:16, default 21
  web:
    generate: true
    image_path: "path/to/image.png"
    background_color: "#hexcode"
    theme_color: "#hexcode"
  windows:
    generate: true
    image_path: "path/to/image.png"
    icon_size: 48 # min:48, max:256, default: 48
  macos:
    generate: true
    image_path: "path/to/image.png"
```

The page number "29/10/1445" is visible in the bottom right corner.

Change Flutter App Icons Flutter ... cont

Step #3: Setup the config file (change the “pubspec.yaml”)

1. Setup the config file

Add your Flutter Launcher Icons configuration to your `pubspec.yaml` or create a new config file called `flutter_launcher_icons.yaml`. An example is shown below. More complex examples [can be found in the example projects](#).

```
dev_dependencies:
  flutter_launcher_icons: ^0.13.1

flutter_launcher_icons:
  android: "launcher_icon"
  ios: true
  image_path: "assets/icon/icon.png"
  min_sdk_android: 21 # android min sdk min:16, default 21
  web:
    generate: true
    image_path: "path/to/image.png"
    background_color: "#hexcode"
    theme_color: "#hexcode"
  windows:
    generate: true
    image_path: "path/to/image.png"
    icon_size: 48 # min:48, max:256, default: 48
  macos:
    generate: true
    image_path: "path/to/image.png"
```

Change Flutter App Icons Flutter ... cont

Step #4: run the package

2. Run the package

After setting up the configuration, all that is left to do is run the package.

```
flutter pub get  
flutter pub run flutter_launcher_icons
```

Change Flutter App Icons Flutter ... cont

```
dependencies:
  flutter: <1 key>

  # The following adds the Cupertino Icons font to your application.
  # Use with the CupertinoIcons class for iOS style icons.
  cupertino_icons: ^1.0.2
  flutter_launcher_icons: ^0.11.0

flutter_icons:
  android: "launcher_icon"
  ios: true
  image_path: "assets/icon.png"
  min_sdk_android: 21 # android min sdk min:16, default 21
```

```
Terminal: Local x + v
Windows PowerShell
Copyright (C) 2016 Microsoft Corporation. All rights reserved.

PS C:\Users\pc\StudioProjects\icon_app> flutter pub get
Running "flutter pub get" in icon_app... 2,384ms

PS C:\Users\pc\StudioProjects\icon_app> flutter pub run flutter_launcher_icons:main
```

```
✓ Successfully generated launcher icons
PS C:\Users\pc\StudioProjects\icon_app>
```

```
dependencies:
  flutter:
    sdk: flutter

  # The following adds the Cupertino Icons font to your application.
  # Use with the CupertinoIcons class for iOS style icons.
  cupertino_icons: ^1.0.6
  flutter_launcher_icons: ^0.13.1

dev_dependencies:
  flutter_test:
    sdk: flutter

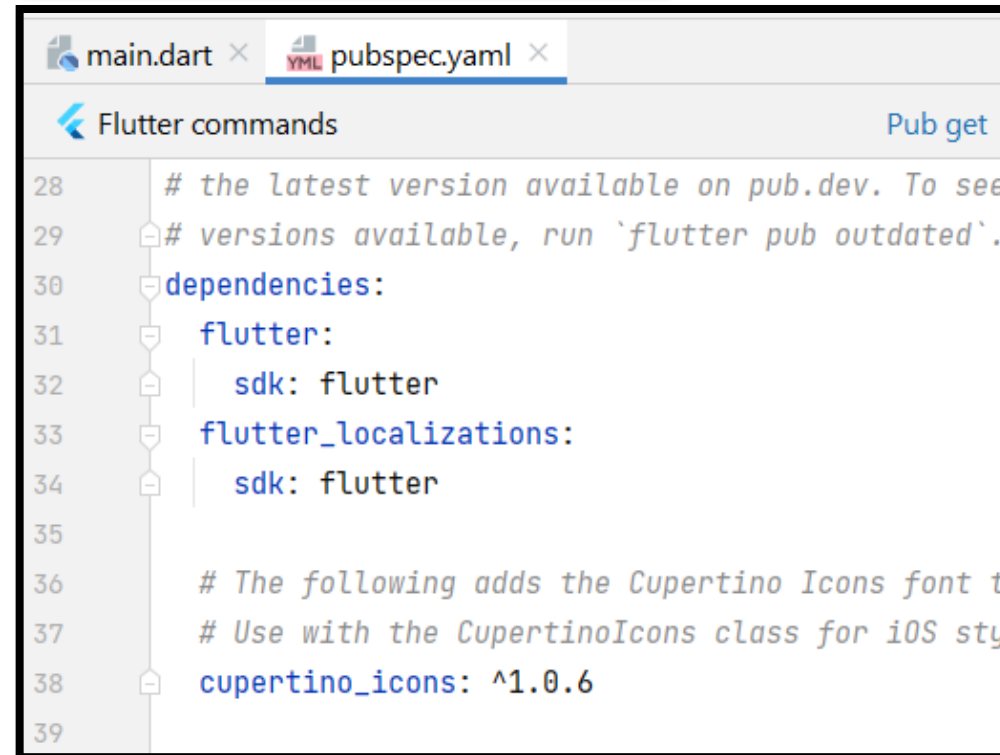
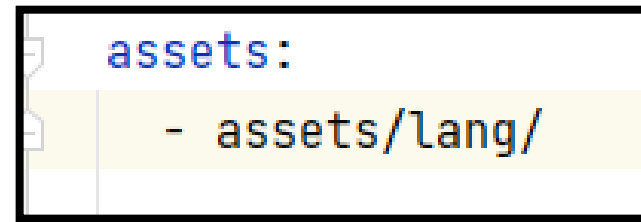
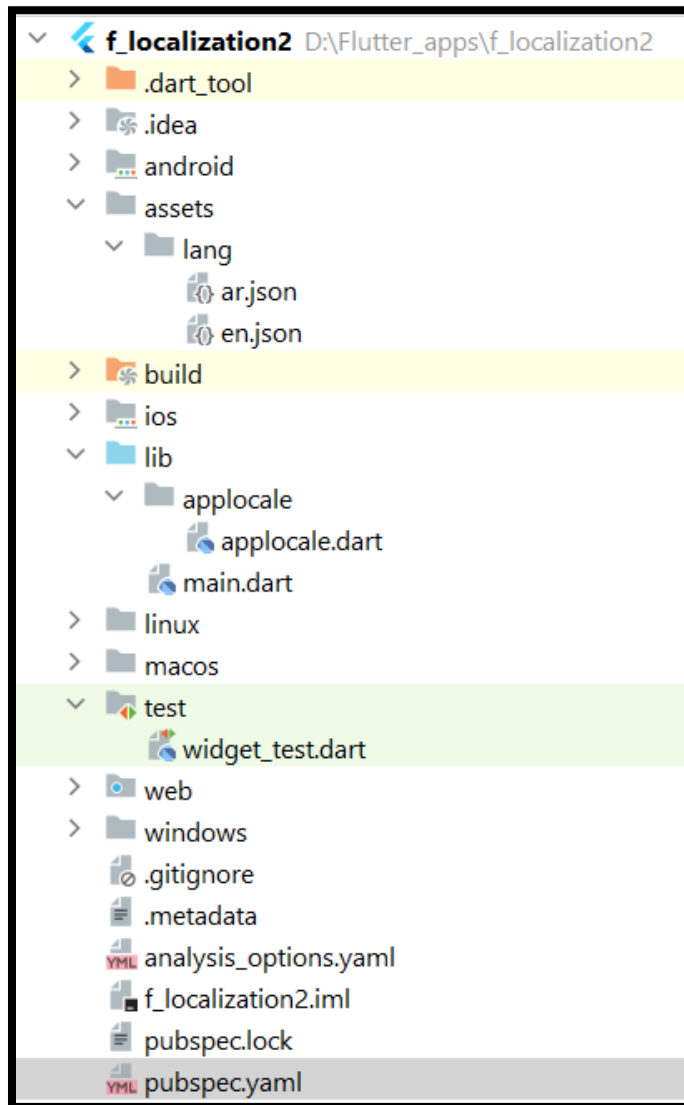
flutter_launcher_icons:
  android: "launcher_icon"
  ios: true
  image_path: "assets/icon/share.png"
  min_sdk_android: 21 # android min sdk min:16, default 21
```

2. Run the package

After setting up the configuration, all that is left to do is run the package.

```
flutter pub get
flutter pub run flutter_launcher_icons
```


Flutter Localization



References:

<https://pub.dev/packages/localization>

<https://pub.dev/documentation/applocale/latest/>

https://github.com/idekhail/f_localization2

Flutter Localization ... main.dart

```
main.dart x
1 import 'package:flutter/material.dart';
2 import 'applocale/applocale.dart';
3 import 'package:flutter_localizations/flutter_localizations.dart';
4
5 void main() {
6   runApp(MyApp());
7 }
8
9 class MyApp extends StatelessWidget {
10   @override
11   Widget build(BuildContext context) {
12     return MaterialApp(
13       title: 'Flutter Localization',
14       theme: ThemeData(
15         primarySwatch: Colors.blue,
16         visualDensity: VisualDensity.adaptivePlatformDensity,
17       ), // ThemeData
18
19       home: MyHomePage(),
20       localizationsDelegates: const [
21         AppLocale.delegate,
22         GlobalMaterialLocalizations.delegate,
23         GlobalWidgetsLocalizations.delegate
24       ],
25       supportedLocales: const [
26         Locale("en", "EN"),
27         Locale("ar", "AR"),
28       ],
29     );
30   }
31 }
```

Flutter Localization ... main.dart

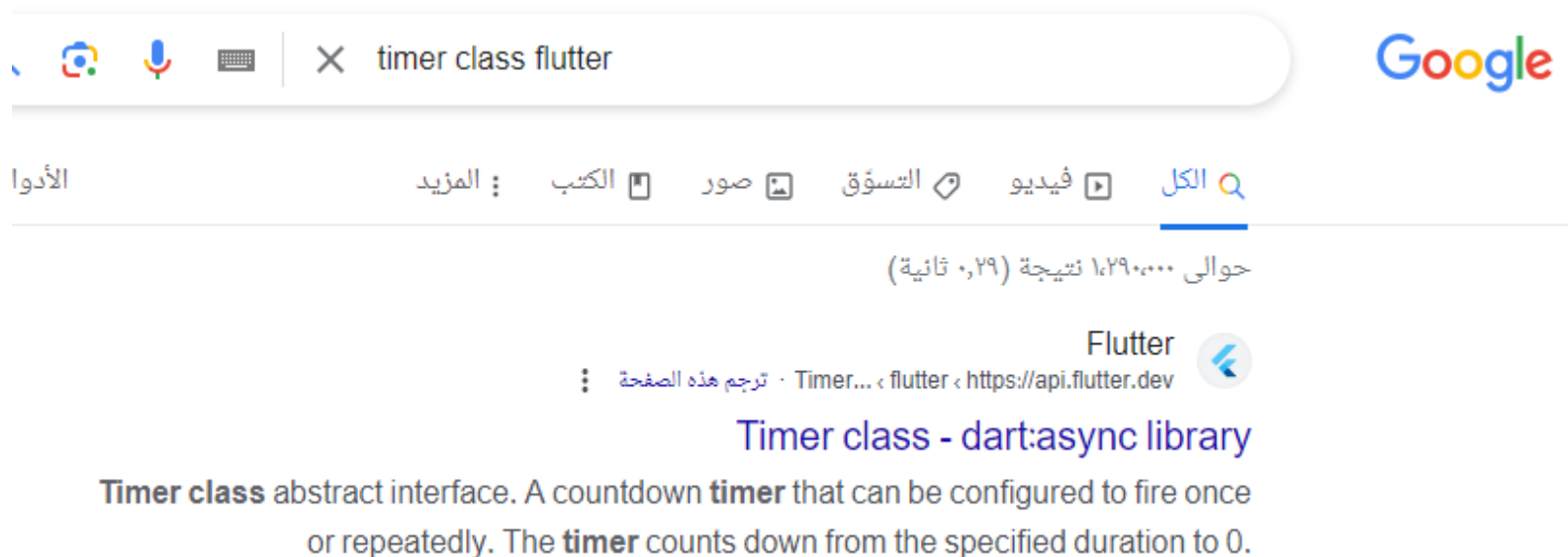
```
30     localeResolutionCallback: (currentLang, supportLang) {
31         if (currentLang != null) {
32             for (Locale locale in supportLang) {
33                 if (locale.languageCode == currentLang.languageCode) {
34                     // mySharedPreferences.setString("lang", currentLang.languageCode) ;
35                     return currentLang;
36                 } } }
37         return supportLang.first;
38     }, ); // MaterialApp
39 } }
40
41 class MyHomePage extends StatelessWidget {
42     @override
43     Widget build(BuildContext context) {
44         return Scaffold(
45             appBar: AppBar( title: Text("${getLang(context, "homepage")}",),),
46             body: Text("${getLang(context, "welcome")}"
47                 ), // Text
48         ); // Scaffold
49     }
66 }
```

Splash Screen

Splash Screen|

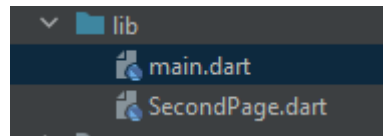
- 1- Timer in Dart Async
- 2- Duration in Dart core
- 3- Navigator.pushAndRemoveUntil

Splash Screen ... Cont



<https://api.flutter.dev/flutter/dart-async/Timer-class.html>

Splash Screen ... Cont (main.dart)



```
main.dart x SecondPage.dart x
1  import 'dart:async';
2  import 'package:flutter/material.dart';
3  import 'package:f_splash_screen/SecondPage.dart';
4  >> void main() {
5      runApp(MyApp());
6  }
7  class MyApp extends StatelessWidget {
8      MyApp({super.key});
9      @override
10     Widget build(BuildContext context) {
11         return MaterialApp(
12             home: MyHomePage(),
13         ); // MaterialApp
14     }
15 }
16 class MyHomePage extends StatefulWidget {
17     MyHomePage({super.key});
18
19     @override
20     _MyHomePageState createState() => _MyHomePageState();
21 }
```

Splash Screen ... Cont

(main.dart)

```
23 class _MyHomePageState extends State<MyHomePage> {
24   void initState() {
25     // TODO: implement initState
26     super.initState();
27     // calling startSplashScreenTimer method, to start the timer
28     startSplashScreenTimer();
29   }
30   // Because we using Timer and it is a Future Object, we used async keyword
31   startSplashScreenTimer() async {
32     var _duration = const Duration(seconds: 5);
33     return Timer(_duration, navigationToNextPage);
34   }
35   void navigationToNextPage() async {
36     Navigator.pushAndRemoveUntil(
37       context, MaterialPageRoute(builder: (context) => SecondPage()),
38       (Route<dynamic> route) => false, //
39     );
40   }
```

Splash Screen ... Cont

(main.dart)

```
42   @override
43   Widget build(BuildContext context) {
44     return Scaffold(
45       backgroundColor: Colors.indigo[300],
46       body: Center(
47         child: Column(
48           mainAxisAlignment: MainAxisAlignment.end,
49           //crossAxisAlignment: CrossAxisAlignment.stretch,
50           children:[
51             const SizedBox(height: 100,),
52             const CircleAvatar( radius: 120, backgroundColor: Colors.white,
53               child: Text('Splash Screen', style:TextStyle(color: Colors.black, fontSize: 20)),
54             ), // CircleAvatar
55             const SizedBox(height: 50,),
56             CircularProgressIndicator( backgroundColor: Colors.pink[300], ),
57             Container(
58               padding: const EdgeInsets.only(top: 20, bottom: 50),
59               child: const Text('My Splash Screen',
60                 style: TextStyle(color: Colors.amber, fontSize: 25),)) // Text, Container
61           ] ), ), );} // Column, Center, Scaffold
62 }
```

Splash Screen ... Cont

(SecondPage.dart)

```
import 'package:flutter/material.dart';
class SecondPage extends StatelessWidget {
  const SecondPage({super.key});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Second Page'),
      ), // AppBar
      body: Column(
        children: [
          const SizedBox(height: 20,),
          Container(
            padding: const EdgeInsets.all(40),
            child: const Text('Build Your Page',
              style: TextStyle(color: Colors.blueGrey, fontSize: 25),)) // Text, Container
        ],
      ), // Column
    ); // Scaffold
  }
}
```


Flutter Animations Comprehensive Guide

- **Introduction**

Animations in a Flutter app can be seen as one of two types:

1. **Drawing-based** animations are animated graphics, vectors, characters, or anything that is “drawn” then animated.
2. **Code-based** animations are focused on widget layouts and styles (lists, colors, text, ..etc).

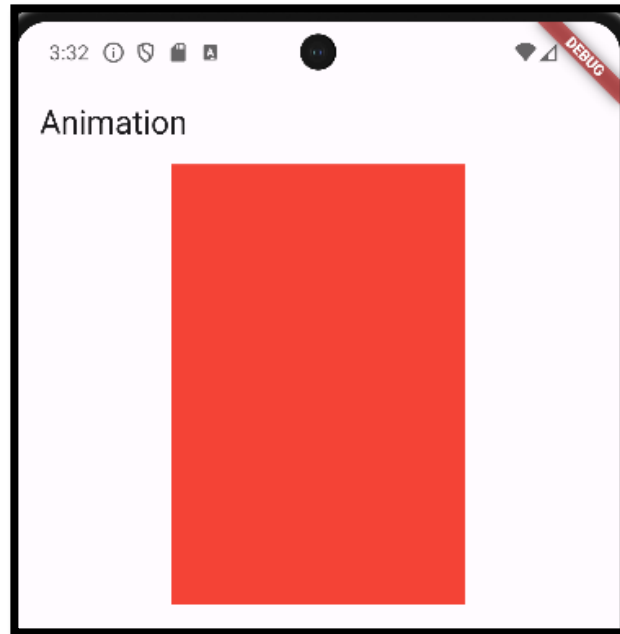
Reference: <https://medium.com/flutter-community/flutter-animations-comprehensive-guide-cb93b246ca5d>

Flutter Animations Comprehensive Guide

Code-based animations in Flutter have two types:

- a. Implicit animations
- b. Explicit animations.

In each of these types, you can either use ready-to-use widgets, or create your own widgets.



```
27 class _MyHomePageState extends State<MyHomePage> {
28   @override
29   Widget build(BuildContext context) {
30     return Scaffold(
31       appBar: AppBar( title: Text('Animation'),),
32       body: Container(
33         child: ListView(
34           children: [
35             InkWell(
36               onTap: (){
37               },
38               child: Center(
39                 child: Container(
40                   color: Colors.red,
41                   width: 200,
42                   height: 300,
43                 ), // Container
44               ), // Center
45             ) // InkWell
46           ],
47         ), // ListView
48       ), // Container
49     ); // Scaffold
50   }
51 }
```

Flutter Animations

a. Implicit animations

- i. Try this code
- ii. Can you try using **Curves** class

<https://api.flutter.dev/flutter/animation/Curves-class.html>

```
child: AnimatedContainer(  
  curve: Curves.bounceIn,  
  decoration: BoxDecoration(  
    color: c,  
    borderRadius: BorderRadius.circular(br)  
  ), // BoxDecoration
```

```
27 class _MyHomePageState extends State<MyHomePage> {  
28   double w = 200.0;  
29   double h = 300.0;  
30   Color c = Colors.red;  
31   double br = 0;  
32   @override  
33   Widget build(BuildContext context) {  
34     return Scaffold(  
35       appBar: AppBar( title: Text('Animation')),  
36       body: Container(  
37         child: ListView(  
38           children: [  
39             InkWell(  
40               onTap: () {  
41                 w = 300.0; h = 400.0; c = Colors.green; br = 100;  
42                 setState(() { });  
43               },  
44               child: Center(  
45                 child: AnimatedContainer(  
46                   decoration: BoxDecoration(  
47                     color: c,  
48                     borderRadius: BorderRadius.circular(br)  
49                   ), // BoxDecoration  
50                   duration: Duration(seconds: 3),  
51                   width: w, height: h,  
52                 ), // AnimatedContainer  
53               ), // Center  
54             ) // InkWell  
55           ],  
56         ), // ListView  
57       ), // Container  
58     ); // Scaffold  
59   }  
60 }
```

Flutter Animations

b. Explicit animations.

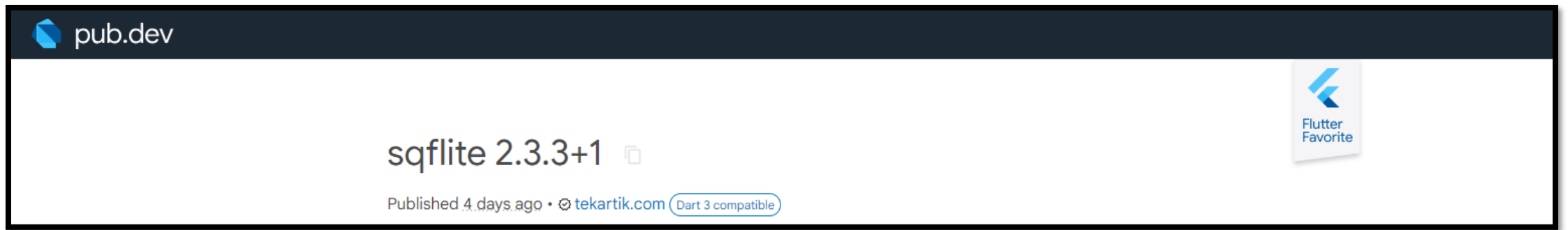
Youtube:

<https://www.youtube.com/watch?v=SZe-XJnw1Bg&list=PL93xoMrxRJIswyhTYGTD3ADy5mJKSkpnh&index=3>

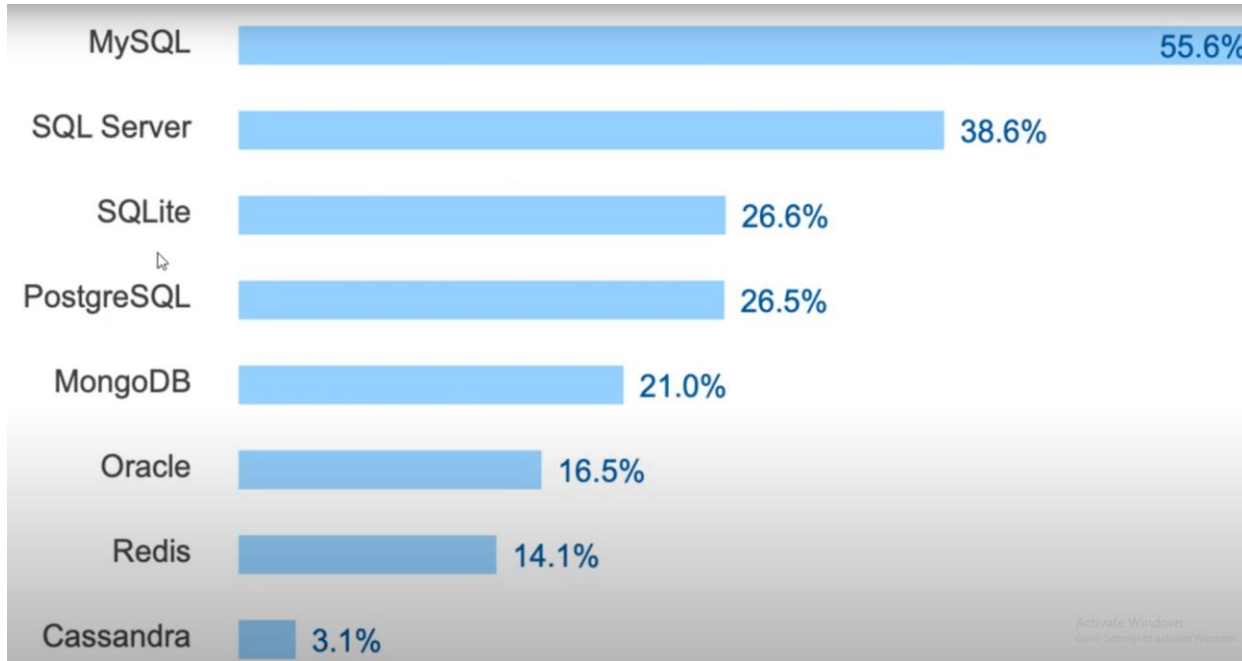
Flutter Animations Comprehensive Guide

Free Animation : <https://lottiefiles.com/featured>

Flutter Database



Using SQFLITE Database With Flutter



SQLite is a Flutter plugin for SQLite databases. It provides a simple, easy-to-use interface to perform standard database operations like CRUD (Create, Read, Update, Delete).

Using SQFLITE Database With Flutter

- ❖ SQLite is a Flutter plugin for SQLite databases.
- ❖ It provides a simple, easy-to-use interface to perform standard database operations like CRUD (Create, Read, Update, Delete).
- ❖ SQLite plugin for [Flutter](#). Supports iOS, Android and MacOS.
 - Support transactions and batches
 - Automatic version management during open
 - Helpers for insert/query/update/delete queries
 - DB operation executed in a background thread on iOS and Android
- ❖ Other platforms support:
 - Linux/Windows/DartVM support using [sqlite_common_ffi](#)
 - Experimental Web support using [sqlite_common_ffi_web](#).
- ❖ Usage example:
 - [notepad_sqlite](#): Simple flutter notepad working on iOS/Android/Windows/linux/Mac

Using SQFLITE Database With Flutter

Getting Started

To get started, you need to add `sqflite` to your project. Follow the steps below:

1. Open the terminal in your project root. You can do this by pressing `Alt+F12` in Android Studio or `Ctrl+`` in VS Code.
2. Run the following command:

```
flutter pub add sqflite
```

This command will add a line to your package's `pubspec.yaml` file and run an implicit `flutter pub get`. The added line will look like this:

```
dependencies:  
  sqflite:
```

Using SQFLITE Database With Flutter

Usage example

Import `sqflite.dart`

```
import 'package:sqflite/sqflite.dart';
```

Opening a database

A SQLite database is a file in the file system identified by a path. If relative, this path is relative to the path obtained by `getDatabasesPath()`, which is the default database directory on Android and the documents directory on iOS/macOS.

```
var db = await openDatabase('my_db.db');
```

There is a basic migration mechanism to handle schema changes during opening.

Many applications use one database and would never need to close it (it will be closed when the application is terminated). If you want to release resources, you can close the database.

```
await db.close();
```

Using SQFLITE Database With Flutter

f_sqflite_db2 / pubspec.yaml

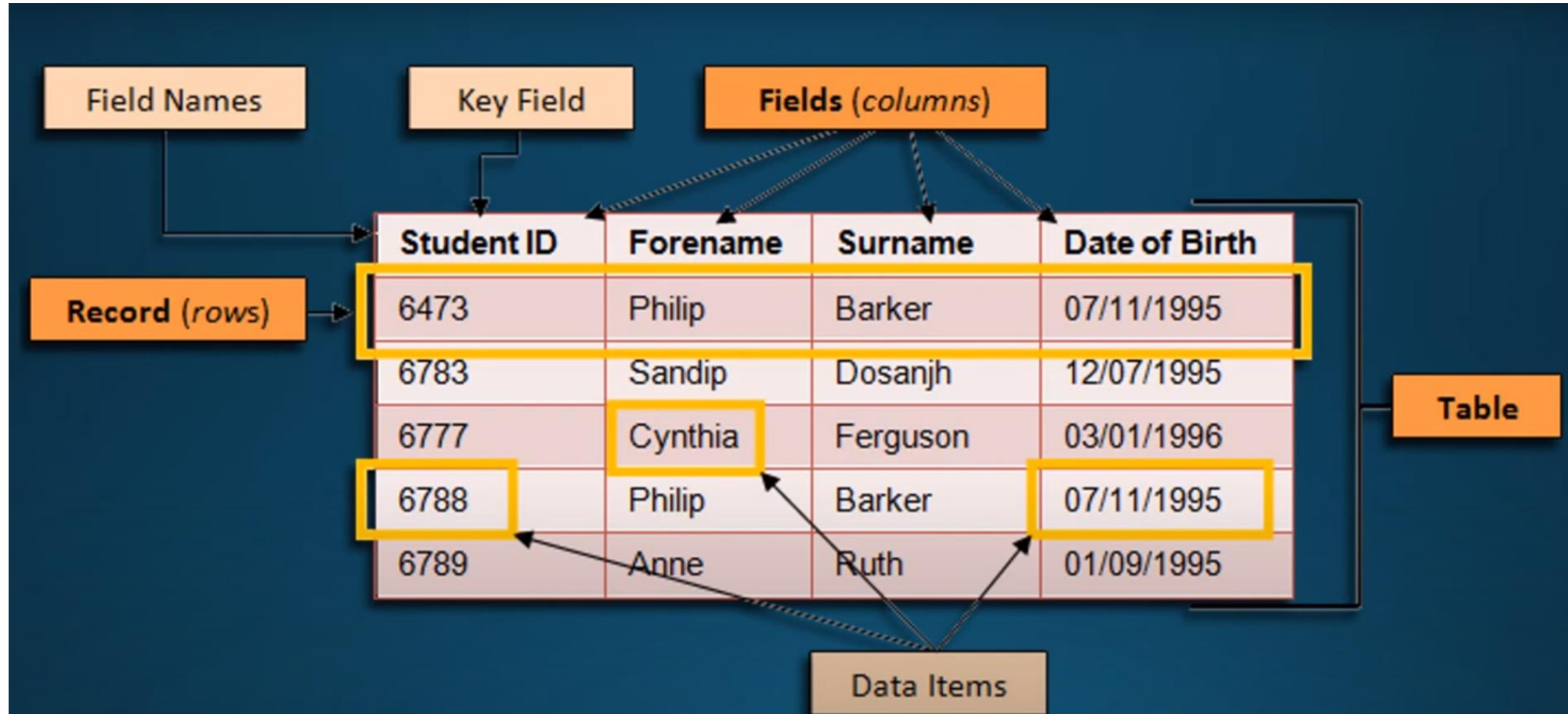
Code

Blame

92 lines (78 loc) · 3.91 KB

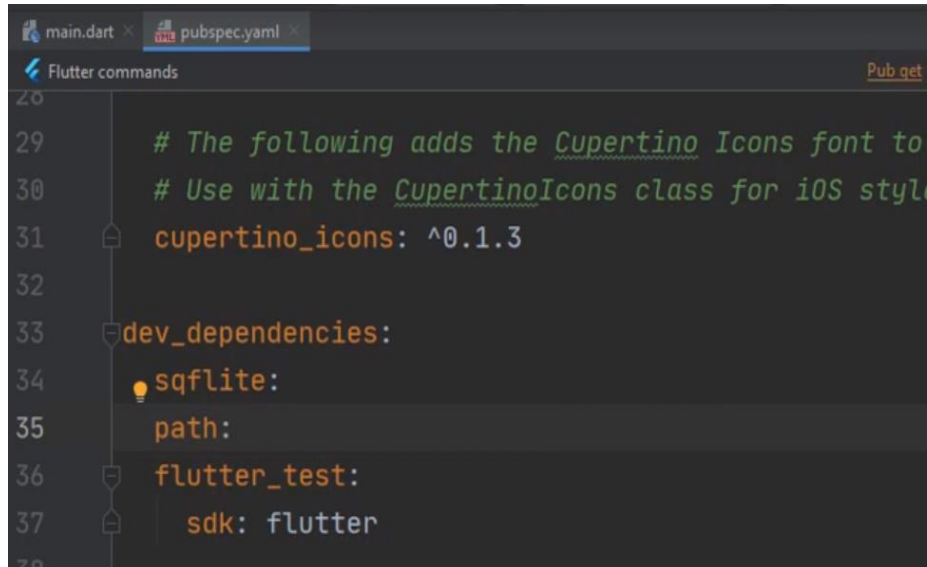
```
29     # versions available, run `flutter pub
30     dependencies:
31       flutter:
32         sdk: flutter
33
34
35     # The following adds the Cupertino
36     # Use with the CupertinoIcons class
37     cupertino_icons: ^1.0.2
38     sqflite: ^2.3.0
39     path: ^1.8.3
40
41     dev_dependencies:
42       flutter_test:
43         sdk: flutter
```

Using SQFLITE Database With Flutter



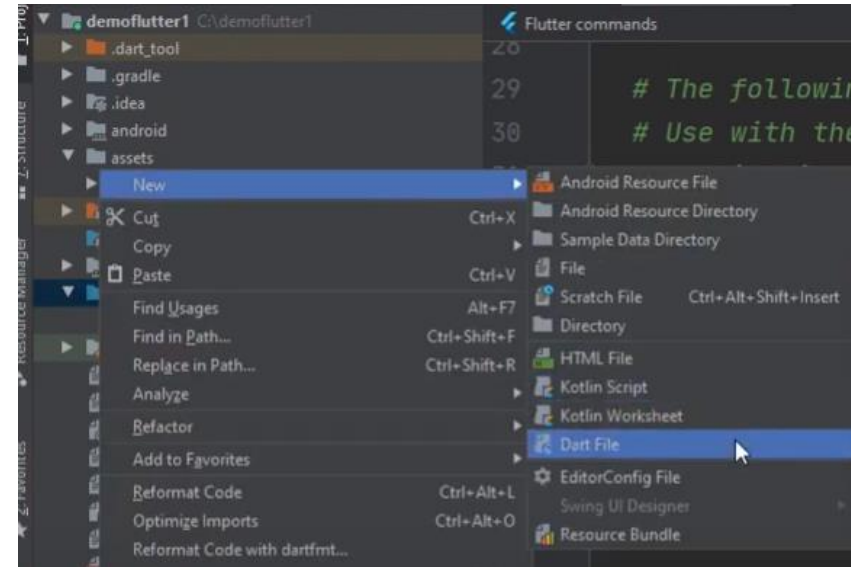
DB example 1: dogs(id, name, age)

1- dependencies

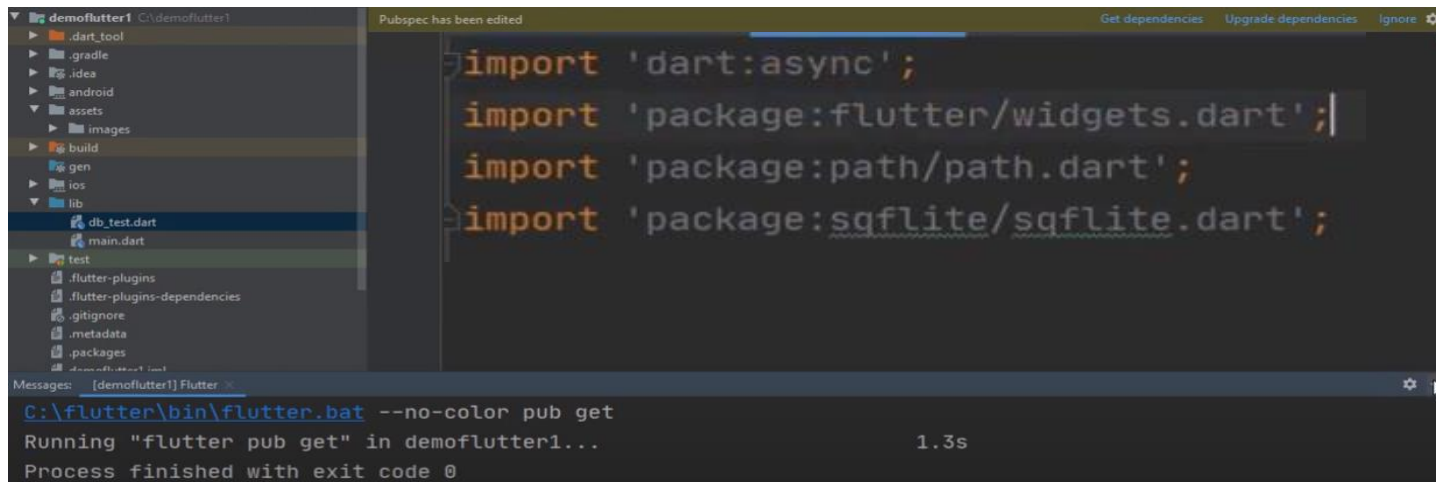


```
20
21
22 # The following adds the Cupertino Icons font to your application.
23 # Use with the CupertinoIcons class for iOS style icons.
24 cupertino_icons: ^0.1.3
25
26 dev_dependencies:
27   sqflite:
28     path:
29   flutter_test:
30     sdk: flutter
```

2- Add db_test.dart file



3- Add libraries



```
import 'dart:async';
import 'package:flutter/widgets.dart';
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';
```

The screenshot shows the 'lib' directory in the IDE with files 'db_test.dart' and 'main.dart'. The 'main.dart' file is open, showing the imports. The bottom status bar shows the command 'C:\flutter\bin\flutter.bat --no-color pub get' and the message 'Running "flutter pub get" in demoflutter1... 1.3s Process finished with exit code 0'.

4. Create the **dogs** table

```
final database = openDatabase(  
    // Set the path to the database. Note: Using the `join` function from the  
    // `path` package is best practice to ensure the path is correctly  
    // constructed for each platform.  
join(await getDatabasesPath(), 'doggie_database.db'),  
    // When the database is first created, create a table to store dogs.  
onCreate: (db, version) {  
    // Run the CREATE TABLE statement on the database.  
    return db.execute(  
        'CREATE TABLE dogs(id INTEGER PRIMARY KEY, name TEXT, age INTEGER)',  
    );  
},  
    // Set the version. This executes the onCreate function and provides a  
    // path to perform database upgrades and downgrades.  
version: 1,  
);
```

5. Insert a Dog into the database

```
class Dog {  
    final int id;  
    final String name;  
    final int age;  
  
    const Dog({  
        required this.id,  
        required this.name,  
        required this.age,  
    });  
  
    // Convert a Dog into a Map. The keys must correspond to the names of the  
    // columns in the database.  
    Map<String, dynamic> toMap() {  
        return {  
            'id': id,  
            'name': name,  
            'age': age,  
        };  
    }  
  
    // Implement toString to make it easier to see information about  
    // each dog when using the print statement.  
    @override  
    String toString() {  
        return 'Dog{id: $id, name: $name, age: $age}';  
    }  
}
```

Using SQFLITE Database With Flutter

- Create ToMap – FromMap Methods
- <https://app.quicktype.io/>

The screenshot shows the Quicktype web application interface. On the left, a sidebar contains a 'Name' field with 'Users' and a 'Source type' dropdown set to 'JSON'. Below this is a JSON input field containing:

```
{
  "id": 1,
  "username": "aaa",
  "password": "123"
}
```

. The main area displays the generated Dart code, which includes imports, a factory method `fromMap`, a `Users` class with properties `id`, `username`, and `password`, and a `toMap` method. On the right, there are two panels of settings. The first panel, titled 'Language', has 'Dart' selected and includes toggle switches for 'Null Safety', 'Types only', 'Put encoder & decoder in Class', 'Make all properties required' (checked), 'Make all properties final' (checked), 'Generate CopyWith method', 'Make all properties optional' (checked), and a 'Copy Code' button. The second panel, titled 'Language Other', shows options for naming conventions, including 'Use this name in `part` directive', 'Use method names fromMap() & toMap()' (checked), 'Generate class definitions with @freezed compatibility', 'Generate annotations for Hive type adapters', and 'Generate' (checked).

quicktype

Sponsor us! Share on Twitter Open on GitHub Options

Name: Users Source type: JSON

```
{
  "id": 1,
  "username": "aaa",
  "password": "123"
}
```

```
// To parse this JSON data, do
//
//   final users = usersFromMap(jsonString);
import 'package:meta/meta.dart';
import 'dart:convert';

Users usersFromMap(String str) => Users.fromMap(json.decode(str));

String usersToMap(Users data) => json.encode(data.toMap());

class Users {
  final int id;
  final String username;
  final String password;

  Users({
    required this.id,
    required this.username,
    required this.password,
  });

  factory Users.fromMap(Map<String, dynamic> json) => Users(
    id: json["id"],
    username: json["username"],
    password: json["password"],
  );

  Map<String, dynamic> toMap() => {
    "id": id,
    "username": username,
    "password": password,
  };
}
```

Language: Dart

☐ Null Safety

☐ Types only

☐ Put encoder & decoder in Class

☒ Make all properties required

☒ Make all properties final

☐ Generate CopyWith method

☒ Make all properties optional

Copy Code

Language: Other

Use this name in `part` directive

Use this name in `part` directive

☒ Use method names fromMap() & toMap()

☐ Generate class definitions with @freezed compatibility

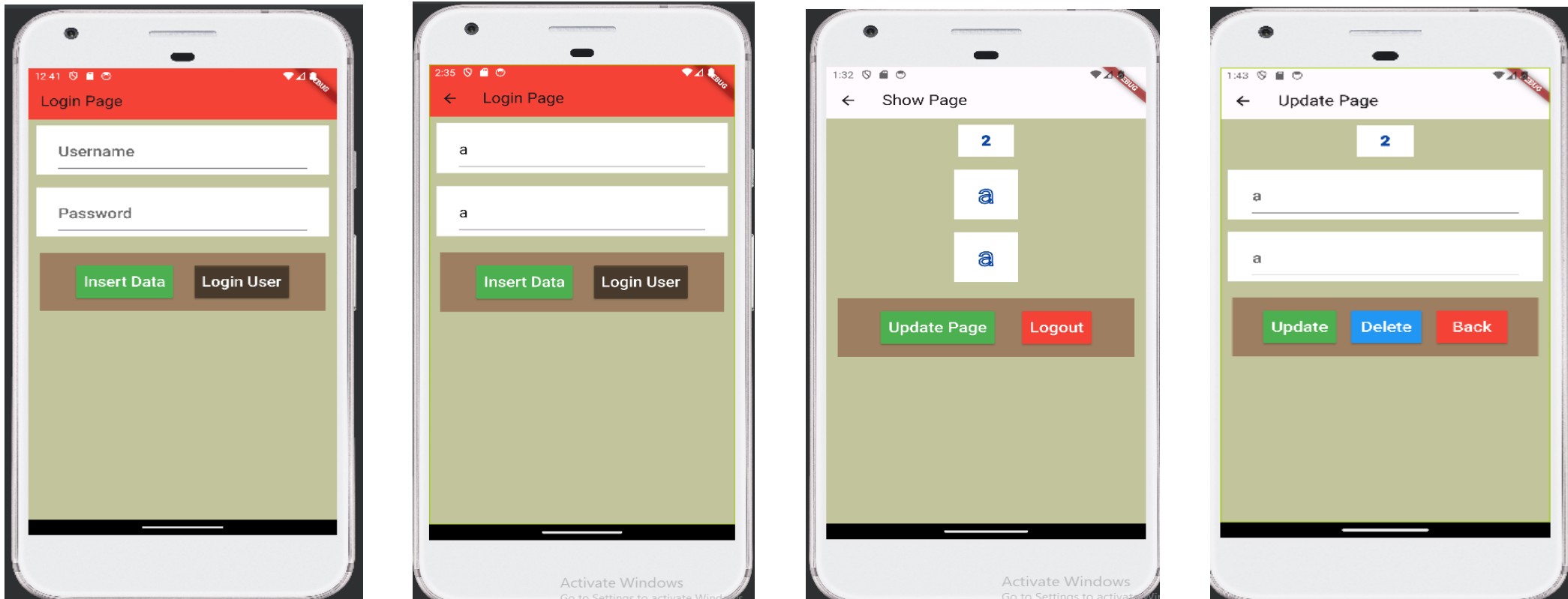
☐ Generate annotations for Hive type adapters

☒ Generate

29/10/1445

88

DB example 2: Design Two Pages



https://github.com/idekhail/f_sqlite_db2

DB example 2: Table Users (Id, Username, Password)

```
users.dart x sqldb.dart x loginpage.dart x
1 // Users Table
2 import 'dart:convert';
3
4 Users usersFromMap(String str) => Users.fromMap(json.decode(str));
5
6 String usersToMap(Users data) => json.encode(data.toMap());
7
8 class Users {
9   final int? id;
10  String username;
11  String password;
12
13  Users({
14    this.id,
15    required this.username,
16    required this.password,
17  });
18
19  factory Users.fromMap(Map<String, dynamic> json) => Users(
20    id: json["id"],
21    username: json["username"],
22    password: json["password"],
23  );
24
25  Map<String, dynamic> toMap() => {
26    "id": id,
27    "username": username,
28    "password": password,
29  };
30 }
```

DB example 2: SQLDB Operation CRUD

```
1 import 'dart:async';
2
3 import 'package:f_sqflite_db2/users.dart';
4 import 'package:path/path.dart';
5 import 'package:sqflite/sqflite.dart';
6
7 class SqlDb{
8
9   final dbName = "userdb2.db";
10
11   String user = ''
12     CREATE TABLE users(
13       id INTEGER PRIMARY KEY AUTOINCREMENT,
14       username TEXT,
15       password TEXT
16     )
17   '';
```

```
18 Future<Database> intialDb() async {
19   final databasepath = await getDatabasesPath();
20   final path = join(databasepath, dbName);
21
22   return openDatabase(path, version: 1, onCreate: (db, version)async{
23     await db.execute(user);
24   });
25 }
```

DB example 2: SQLDB Operation CRUD

```
27 Future<Users> loginUser(Users usr) async{
28     final Database db = await intialDb();
29     final result = await db.rawQuery(
30         "select * from users where username = '${usr.username}' AND password = '${usr.password}'");
31     if (result.isNotEmpty) {
32         return Users.fromMap(result.first);
33     } else {
34         throw Exception("username or password not found");
35     }
36 }

37
38 Future<int> createUser(Users usr) async{
39     final Database db = await intialDb();
40     return db.rawQuery("INSERT INTO 'users' ('username', 'password') VALUES ('${usr.username}', '${usr.password}')");
41 }
```

DB example 2: SQLDB Operation CRUD

```
42 Future<int> deleteUser(Users usr) async{
43     final Database db = await initialDb();
44     return await db.rawQuery('DELETE FROM users where id =?', ['${usr.id}']);
45 }
46 Future<int> updateUser(Users usr) async{
47     final Database db = await initialDb();
48     return await db.rawQuery(
49         "UPDATE users SET username = \`${usr.username}\`, password = \`${usr.password}\` WHERE id = ${usr.id}"
50     );
51 }
52 // A method that retrieves all the users from the users table.
53 Future<List<Users>> getAllUsers() async {
54     final Database db = await initialDb();
55     List<Map> list = await db.rawQuery('SELECT * FROM users');
56     return list.map((usr) => Users.fromMap(usr as Map<String, dynamic>)).toList();
57 }
58 }
```

DB example 2: main → Login Page

```
loginpage.dart x main.dart x
1  import 'package:f_sqflite_db2/loginpage.dart';
2  import 'package:flutter/material.dart';
3
4  >> void main() async{
5      runApp( MyApp());
6  }
7
8  class MyApp extends StatelessWidget {
9      @override
10     Widget build(BuildContext context) {
11         return MaterialApp(
12             title: 'Sqflite DB',
13
14             home: LoginPage(),
15         ); // MaterialApp
16     }
17 }
```

DB example 2: Login Page

```
loginpage.dart x
1 import 'package:f_sqflite_db2/showpage.dart';
2 import 'package:f_sqflite_db2/sqladb.dart';
3 import 'package:f_sqflite_db2/users.dart';
4 import 'package:flutter/material.dart';
5
6 class LoginPage extends StatefulWidget{
7   @override
8   State<LoginPage> createState()=> _HomeState() ;
9 }
10
11 class _HomeState extends State<LoginPage>{
12   final userController = TextEditingController();
13   final passController = TextEditingController();
14   bool isChecked = false;
15   bool isLoginTrue = false;
16   final db = SqlDb();
17 }
```

```
18 create() async{
19   var response = await db.createUser(Users(username: userController.text,
20     password: passController.text));
21   if(response > 0){
22     if(!mounted) return;
23     print("ok");
24     const AlertDialog(title: Text("Ok"));
25   }else{
26     print("Error");
27     const AlertDialog(title: Text("Error"));
28   }
29 }
30 login() async{
31   var user = await db.loginUser(Users(username: userController.text,
32     password: passController.text));
33   if(user != null){
34     if(!mounted) return;
35     Navigator.push(context, MaterialPageRoute(builder: (context)=>ShowPage(usr: user)));
36   }else{
37     setState(() {
38       isLoginTrue = true;
39       const AlertDialog(title: Text("Error", style: TextStyle(backgroundColor: Colors.green),));
40     });
41   }
42 }
```

DB example 2: Button (Create, Login)

```
43
44 @override
45 Widget build(BuildContext context) {
46   return Scaffold(
47     appBar: AppBar(
48       backgroundColor: Colors.red,
49       title: const Text('Login Page'),
50     ), // AppBar
51     body: Container(
52       color: const Color.fromARGB(100, 100, 100, 109, 5),
53       child: Column(
54         children: [
55           Container(
56             color: const Color.fromRGB0(255, 255, 255, 1),
57             margin: const EdgeInsets.all(10),
58             padding: const EdgeInsets.symmetric(horizontal: 30, vertical: 10),
59             child: TextFormField(
60               controller: userController,
61               style: const TextStyle(fontSize: 22),
62               decoration: const InputDecoration(hintText: 'Username'),
63             ), // TextFormField
64           ), // Container
65           Container(
66             color: const Color.fromRGB0(255, 255, 255, 1),
67             margin: const EdgeInsets.all(10),
68             padding: const EdgeInsets.symmetric(horizontal: 30, vertical: 10),
69             child: TextFormField(
70               controller: passController,
71               style: const TextStyle(fontSize: 22),
72               decoration: const InputDecoration(hintText: 'Password'),
73             ), // TextFormField
74           ), // Container
```

```
76
77   Container(
78     margin: const EdgeInsets.all(15),
79     color: const Color.fromARGB(100, 100, 100, 19, 5),
80     padding: const EdgeInsets.all(20),
81     child: Row(
82       mainAxisAlignment: MainAxisAlignment.spaceEvenly,
83       children: [
84         //const Padding(padding: EdgeInsets.all(30)),
85         MaterialButton(
86           padding: const EdgeInsets.all(10),
87           color: Colors.green,
88           textColor: Colors.white,
89           onPressed: () async{
90             create();
91           },
92           child: const Text("Insert Data",
93             style: TextStyle(fontSize: 22),
94           ), // Text
95         ), // MaterialButton
96         // Spacer(flex: 1),
97         MaterialButton(
98           padding: const EdgeInsets.all(10),
99           color: Colors.black38,
100          textColor: Colors.white,
101          onPressed: () async{
102            login();
103          },
104          child: const Text("Login User",
105            style: TextStyle(fontSize: 22),
106          ), // Text
107        ), // MaterialButton
```


DB example 2: Show Page

```
showpage.dart x
1 import 'package:f_sqflite_db2/loginpage.dart';
2 import 'package:f_sqflite_db2/sqlldb.dart';
3 import 'package:f_sqflite_db2/updatepage.dart';
4 import 'package:f_sqflite_db2/users.dart';
5 import 'package:flutter/material.dart';
6
7 class ShowPage extends StatelessWidget {
8   final db = SqlDb();
9   final userController = TextEditingController();
10  final passController = TextEditingController();
11  Users usr;
12  @override
13
14  ShowPage({required this.usr});
```

```
16 @override
17 Widget build(BuildContext context) {
18   return Scaffold(
19     appBar: AppBar(
20       title: const Text('Show Page'),
21     ), // AppBar
22     body: Container(
23       color: const Color.fromARGB(100, 100, 100, 109, 5),
24       child: Column(
25         children: [
26
27           Container(
28             color: const Color.fromRGBO(255, 255, 255, 1),
29             margin: const EdgeInsets.all(10),
30             padding: const EdgeInsets.symmetric(horizontal: 30, vertical: 10),
31             child: Text(
32               '${usr.id}',
33               style: TextStyle(
34                 fontSize: 20,
35                 foreground: Paint()
36                   ..style = PaintingStyle.stroke
37                   ..strokeWidth = 2
38                   ..color = Colors.blue[900]!,
39               ), // TextStyle
40             ), // Text
41           ), // Container
```

DB example 2: Show Page

```
43  Container(  
44  color: const Color.fromRGBO(255, 255, 255, 1),  
45  margin: const EdgeInsets.all(10),  
46  padding: const EdgeInsets.symmetric(horizontal: 30, vertical: 10),  
47  child: Text(  
48    '${usr.username}',  
49    style: TextStyle(  
50      fontSize: 40,  
51      foreground: Paint()  
52        ..style = PaintingStyle.stroke  
53        ..strokeWidth = 2  
54        ..color = Colors.blue[900]!,  
55    ), // TextStyle  
56  ), // Text  
57 ), // Container
```

DB example 2: Show Page

```
85   MaterialButton(  
86     padding: const EdgeInsets.all(10),  
87     color: Colors.green,  
88     textColor: Colors.white,  
89     onPressed: () async {  
90       Navigator.push(context, MaterialPageRoute(  
91         builder: (context)=>UpdatePage(usr: usr)));  
92     },  
93     child: const Text(  
94       "Update Page",  
95       style: TextStyle(fontSize: 22),  
96     ), // Text  
97   ), // MaterialButton
```

DB example 2: Update Page

```
showpage.dart x updatepage.dart x
1 import 'package:f_sqflite_db2/loginpage.dart';
2 import 'package:f_sqflite_db2/showpage.dart';
3 import 'package:f_sqflite_db2/sqldb.dart';
4 import 'package:f_sqflite_db2/users.dart';
5 import 'package:flutter/material.dart';
6
7
8 class UpdatePage extends StatelessWidget {
9   final db = Sqldb();
10  final userController = TextEditingController();
11  final passController = TextEditingController();
12  Users usr;
13  @override
14  UpdatePage({required this.usr});
15
16  updateU() async{
17    usr.username = userController.text.toString();
18    usr.password = passController.text.toString();
19    // print(this.usr.username);
20    var response = await db.updateUser(usr);
21    if(response > 0){
22      print("Ok");
23    }else{
24      print("Error");
25    }
26  }
```

```
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
Container(
  color: const Color.fromRGBO(255, 255, 255, 1),
  margin: const EdgeInsets.all(10),
  padding: const EdgeInsets.symmetric(horizontal: 30, vertical: 10),
  child: Text(
    '${usr.id}',
    style: TextStyle(
      fontSize: 20,
      foreground: Paint()
        ..style = PaintingStyle.stroke
        ..strokeWidth = 2
        ..color = Colors.blue[900]!,
    ), // TextStyle
  ), // Text
), // Container

Container(
  color: const Color.fromRGBO(255, 255, 255, 1),
  margin: const EdgeInsets.all(10),
  padding: const EdgeInsets.symmetric(horizontal: 30, vertical: 10),
  child: TextFormField(
    controller: userController,
    style: const TextStyle(fontSize: 22),
    decoration: InputDecoration(hintText: '${usr.username}'),
  ), // TextFormField
), // Container
```

Complete the project by doing the next steps

1. displaying a pop-up message when a button is clicked
2. Create Non Duplicate User Name in Database
3. Show all Users in DB.

1. Flutter Toast Package

1. First, you need to install the `fluttertoast` package of Flutter. To do this, add the following code to your `pubspec.yaml` file and run the `flutter packages get` command:

```
dependencies:  
  fluttertoast: ^8.0.7
```

2. Next, create an `onPressed` function to display the message box when the button is clicked. In this function, you can use the `Fluttertoast.showToast()` function to display the message box. You can specify the content, position, duration, and style of the message box. In the example code below, the message box will display "Hello, World!":

```
body: Center(  
  child: ElevatedButton(  
    onPressed: () {  
      Fluttertoast.showToast(  
        msg: 'Hello, World!',  
        toastLength: Toast.LENGTH_SHORT,  
        gravity: ToastGravity.CENTER,  
        timeInSecForIosWeb: 1,  
        backgroundColor: Colors.grey,  
        textColor: Colors.white,  
        fontSize: 16.0);  
    },  
    child: Text('Show Pop-Up Message'),  
  ),
```

The End

Top Flutter interview questions and answers in 2024 - 2025 From different links

References:

1. <https://www.interviewbit.com/flutter-interview-questions/>
2. <https://www.turing.com/interview-questions/flutter>

1. Is Flutter an SDK?

Yes, Flutter is an SDK (Software Development Kit).

2. What are the Reasons for Using Flutter?

Using Flutter to create mobile apps has several benefits:

- **Less coding time.**
- **Cross-platform development.**
- **Performs like a Native App.**
- **Minimal Code.**
- **Customized designs.**

3. What is Flutter Architecture?

Flutter architecture comprises three layers:

- **Upper layer-** The platform built using Dart that handles app widgets, gestures, animations, drawings, and materials.
- **Flutter engine-** Manages the display and text formatting.
- **Built-in service:** Manages event loops, packages, and plugins.

4. What are the different build modes in Flutter?

- **Debug mode:** Switch to debug mode when using hot reload during development.
- **Profile mode:** Use profile mode when you wish to evaluate performance.
- **Release mode:** Use release mode when your app is ready for distribution.

5. Which technology is used by Flutter?

C, C++ the 2D rendering engine Skia, and the object-oriented language Dart are all used to create Flutter.

6. Name some popular apps that use Flutter.

It is used in the development of several popular apps. Some of the apps are:

Tencent – Alibaba – Reflect - Google Ads - Birch Finance

7. What is Dart?

An object-oriented programming language similar to the C Language is called Dart.

In 2011, Google created this open-source language.

The language was designed specifically for front-end coding of mobile applications.

8. What do Flutter packages and plugins do?

Packages are

groups of related classes,
interfaces,
and sub-packages,
whether Flutter or any other object-oriented language.

The usage of plugins and packages helps developers write code more quickly.

Additionally, the coder can use packages and plugins to avoid writing the code from scratch for every task.

The differences between plugins and packages are negligible.

A package is a new component or code written in Dart.

In contrast, plugins use native code to enable extra functionality.

Despite their slight differences, the two names are occasionally used interchangeably.

9. Which is better, Flutter or React?

- While Google first introduced the Flutter framework, Facebook developed React Native.
- Therefore, both frameworks have excellent features and communities.
- While React Native builds its apps using JavaScript, Flutter employs the Dart programming language.
- The native hybrid app is created using Flutter and React Native from a single codebase.
- Both the iOS and Android platforms support these apps.

10. What are the points of difference between runApp() and main() in Flutter?

Main()

When rendering the widget tree on the screen, it returns the widgets connected to the screen as the widget tree's root.

Primary function refers to the function that drives the app

RunApp()

It is a feature that is used to launch the software.

You cannot create any program in Flutter without the main() function.

11. Explain the Primary Axis and Cross Axis Alignment

Our preferences determine how a row and column widget aligns with its children using the CrossAxisAlignment and PrimaryAxisAlignment.

- **PrimaryAxisAlignment**

In PrimaryAxisAlignment, the columns run vertically while the rows run horizontally.

- **CrossAxisAlignment**

In CrossAxisAlignment, the columns run horizontally while the rows run vertically.

What does SDK stands for?

- Software Data Kit
- Software Database Kit
- **Software Development Kit**
- None of the above

Choose the best editors for Flutter app development.

- VS Code
- Sublime Text
- Android Studio
- **All of the above**

What are the type of tests, that we can perform in flutter?

- Unit test
- Widget test
- Integration test
- **All of the above**

References:

<https://medium.com/@jaiswalsatyam/flutter-mcq-test-b703afb01d24>

What is the difference between StatelessWidget and StatefulWidget in Flutter?

In Flutter, StatelessWidget and StatefulWidget are two different types of widgets that are used to create UI elements.

- StatelessWidget is a widget that does not have any mutable state. Once it is built, its properties cannot be changed. Examples of StatelessWidget include Text, Icons, and Images.
- StatefulWidget is a widget that has a mutable state. It can change its properties based on user interactions. Examples of StatefulWidget include TextField, Checkbox, and Slider.

References:

<https://www.geeksforgeeks.org/flutter-interview-questions/>

Flutter is developed by

- Google
- Meta
- Microsoft
- Twitter

Which of the following language is used to build flutter app?

- Go
- Java
- Dart
- React

References:

<https://www.linkedin.com/pulse/flutter-multiple-choice-questions-onlineinterviewquestions>

1. How many types of widgets are there in Flutter?

- A. 2
- B. 4
- C. 6
- D. 8+

Answer: A. There are two types of widgets available to developers in Flutter. These are stateful and stateless widgets.

2. When building for iOS, Flutter is restricted to an ____ compilation strategy

- A. AOT (ahead-of-time)
- B. JIT (Just-in-time)
- C. Transcompilation
- D. Recompilation

Answer: A. Due to iOS restrictions on dynamic code execution Flutter apps must use AOT compilation on the iOS platform.

References:

<https://pangea.ai/resources/flutter-mcq-questions-and-answers>